

DeepSeek Harness 白皮书 • dsh-handbook

从 0 到 1 玩转 DeepSeek Harness 的新手百科全书。★ 如果你觉得有帮助，点个 Star 支持持续更新 · 中文 · [English](#)

DeepSeek Harness 白皮书

从 0 到 1 玩转官方开源 Agent 运行时 · 新手百科全书

dsh-handbook

7 章 · 中英双语 · PDF
benchmark · demo · 插件实战

dsh-handbook 白皮书 章节 14 PDF 3.9MB license CC-BY-NC-SA-4.0 dsh 0.1.0-rc.6

这是什么

DeepSeek Harness (dsh) 是 DeepSeek 官方 2026-08-13 开源的 Agent 运行时——一个"一切皆插件" (everything is a plugin) 的框架。



但官方文档以架构说明为主，缺少一条从零上手的路径。

这本白皮书补上这条路：从"什么是 Agent 运行时"讲起，到安装、使用、开发插件、性能调优——每一章都有可复制、可运行的命令，全部在本机实测验证。**目标是：任何一个开发者，跟着这本书都能从 0 到 1 用起来、写起来。**

快速体验 (30 秒)

```
# 1. 安装 (需要 Node.js ≥ 22)
npx -y @deepseek-ai/dsh web

# 2. 浏览器打开 http://127.0.0.1:3080，开始对话
# 3. 或跑一次性任务 (适合脚本/CI)
dsh --profile headless "你好，请用一句话介绍自己"
```

想系统学？看 [📖 学习路径 \(3 天计划\)](#)；想先跑？[第 2 章：五分钟快速上手](#)

目录 (从 0 到 1)

#	章节	你将学会	状态
	学习路径 (3 天计划)	从 0 到 1: 每天目标 + 验收标准 + 学习原则	✓
1	认识 DeepSeek Harness · EN	它是什么、为什么值得学、与主流 Agent 全面对比、FAQ	✓ 双语
2	五分钟快速上手 · EN	安装、web/headless 双模式、模型与推理档位、排障	✓
3	profile 与插件系统	可定制骨架、插件挂载、host/client 双半、扩展点、真实坑	✓
4	插件开发实战	从零写第一个插件 (完整代码 + 测试 + 实机验证)	✓
5	实战案例	三个真实开源 PR 的完整闭环	✓
6	进阶与性能调优	推理档位策略、耗时分析、踩坑清单	✓
7	生态与资源	官方入口、参与路径、阅读建议	✓
8	工具与上下文系统	60+ 能力包地图、内置工具、上下文注入、compaction、安全模型	✓
9	MCP、子代理与工作流	外部工具接入、并行子代理、多步编排、Agent 系统蓝图	✓
10	复杂实战案例	dsh 真实跑出的 : 数据清洗+可视化管线 (186s)、5-bug 修复 +49 测试 (94s), 含产物与判断力分析	✓
11	未来展望	技术/生态/竞争/行业/机会/风险 六角度演进预测 + 时间线	✓
附	术语表与命令速查 · Benchmark	30+ 术语、命令速查、3 Agent 实测	✓

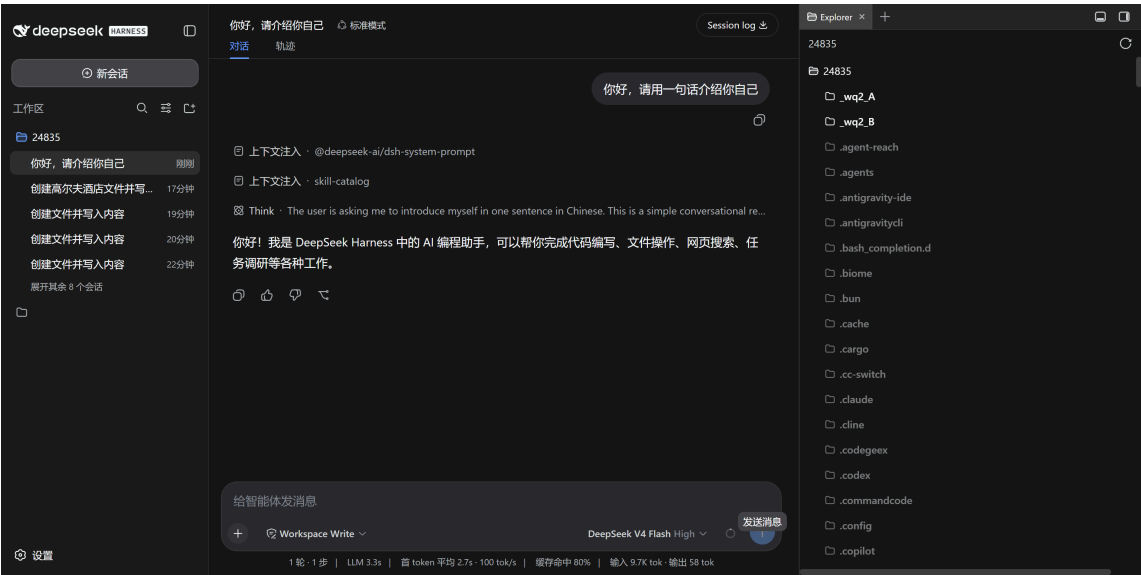
内容精华速览 (点开即看, 不止链接)

- ▶  第 1 章: 认识 DeepSeek Harness —— 三个直觉 + 能力矩阵
- ▶  第 2 章: 五分钟快速上手 —— 30 秒跑起来
- ▶  第 3 章: profile 与插件系统 —— 可定制骨架
- ▶  第 4 章: 插件开发实战 —— 完整可运行代码
- ▶  第 5 章: 实战案例 —— 三个真实开源 PR 的完整闭环
- ▶  第 6 章: 进阶与性能调优 —— 时间花在哪
- ▶  第 8 章: 工具与上下文系统 —— 能力引擎
- ▶  第 9 章: MCP、子代理与工作流 —— Agent 系统化
- ▶  第 10 章: 复杂实战案例 —— dsh 真实跑出来的
- ▶  附录: 术语表 + 命令速查
- ▶  第 7 章: 生态与资源 —— 加入 dsh 生态的地图

演示 (Demo) —— 直接看效果

① Web UI 对话 (dsh web)

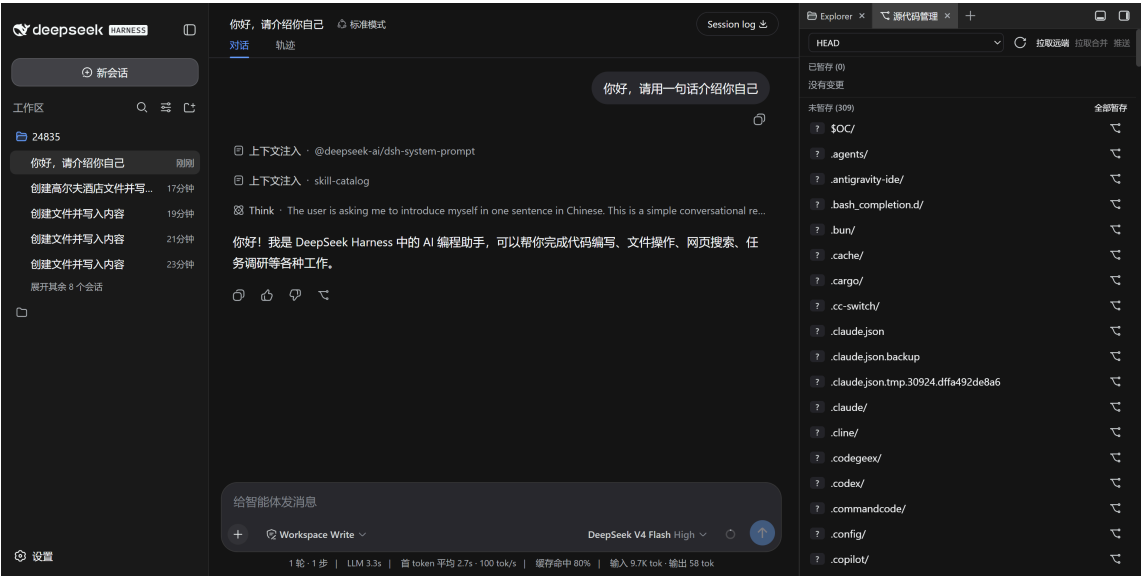
dsh web # → http://127.0.0.1:3080



② Headless CLI（一次性任务，适合脚本/CI）

```
dsh --profile headless "你好, 请用一句话介绍你自己"
# → 你好! 我是 DeepSeek 驱动的 AI 编程助手, 可以帮你写代码、调试问题、
#     处理文件、搜索资料, 以及完成各种开发和办公任务。
```

③ 插件生态（Git 面板， dsh-better-sidebar ）



完整图文演示见 [📺 30 秒看懂 dsh。](#)

DSH vs 主流 Agent（能力矩阵）

维度	dsh	Claude Code	OpenAI Codex	OpenCode	Gemini CLI	Kimi CLI
----	-----	-------------	--------------	----------	------------	----------

开源	✔ MIT	✘	✘	✔ MIT	✘	✘
模型绑定	模型无关	Claude 系	GPT 系	任意	Gemini 系	Kimi 系
插件体系	官方级：一切皆插件，60+ 官方包	配置/钩子	配置	配置	无	无
自定义界面	✔ (client 半)	✘	✘	部分	✘	✘
自动化/CI	✔ headless	✔	✔	✔	✔	✔
TUI	插件可做	✔ 内置	✔ 内置	✔ 内置	✔	✔
生态阶段	零日 (2026-08-13)	成熟	成熟	成熟	成熟	早期
适合谁	深度定制+生态	开箱即用	开箱即用	OpenCode 用户	Google	Kimi

实测案例、同模型多 Agent 对比数据见 [第 1 章](#) 与 [benchmark](#) 章节。

同模型 × 不同 Agent 实测 (2026-08-13)

模型统一 `deepseek-v4-flash` (同一网关、同一 key) , 只对比 Agent 工程层。3 任务全部正确完成, 差异在效率:

Agent	总耗时	正确率
omp	36s	27/27 ✔
dsh	85s	27/27 ✔
opencode	114s	27/27 ✔

3 轮采样中位数, 27/27 全对。完整方法/解读见 [📖 Benchmark 附录](#)。

白皮书 PDF

- 中文完整版: [DeepSeek-Harness-白皮书.pdf](#) (13 章节, 109k 字符, 3.9MB)
- 英文完整版: [DeepSeek-Harness-Handbook.pdf](#) (10 章, 54k 字符)

为什么值得读 (而不是只看官方文档)

官方文档	本白皮书
架构视角 (AGENTS.md / architecture.md)	新手视角: 一条从 0 到 1 的路径
零散示例	每章可运行, 命令全部实测
无中文教程	中文优先, 英文同步
无生态实操	真实插件/PR 拆解 (含踩坑与安全约束)

与生态联动

本白皮书的方法论来自真实开源实践：

- [dsh-tool-turbo](#) —— 工具调用提速插件（第 4/6 章源码）
- [DSH-better-sidebar](#) —— 社区侧边栏插件（第 5 章案例）

贡献与反馈

- 章节/命令失效？rc 版本迭代所致，欢迎 issue 指正
- 想参与？见 [第 7 章：生态与资源](#)

学习路径：从 0 到 1 的 3 天计划

承诺兑现：任何开发者，3 天从零到能开发插件。本路径把 9 章拆成 3 天，每天有明确目标与验收标准。

Day 1：理解 + 跑起来（第 1-2 章）

目标：理解 dsh 是什么，能独立启动、对话、配置。

时间	任务	验收
30 分钟	读第 1 章（三个直觉 + 能力矩阵）	能向别人解释"dsh 和 Claude Code 的区别"
30 分钟	第 2 章动手：dsh web 对话 + headless	能启动 UI 完成一次对话；能跑一条 headless 命令
30 分钟	配置模型/推理档位 + 排障速查	能改 settings.yaml；能解决端口占用

验收：`dsh --profile headless "你好"` 有回复。

Day 2：理解骨架 + 第一个插件（第 3-4 章）

目标：理解 profile/插件机制，写出第一个能跑的插件。

时间	任务	验收
1 小时	第 3 章（profile、挂载、host/client、扩展点、坑）	能挂载社区插件（Git 面板）
1.5 小时	第 4 章抄写 tool-turbo：纯函数 + agent/request 注入	插件加载无错，日志出现决策输出

验收：自己的插件在 dsh 里生效（日志证明注入发生）。

Day 3：实战 + 调优（第 5-6 章）

目标：能评估插件质量、调优性能、理解生态玩法。

时间	任务	验收
1 小时	第 5 章三个真实 PR（安全红线、沙箱约束、测试方法）	能说出"写一个 PR 的完整闭环"
1 小时	第 6 章性能模型（思考占 90%）+ 档位策略 + 坑	能给一个任务选档位、预估瓶颈
30 分钟	第 8-9 章扫读（工具/上下文、MCP/子代理/工作流）+ 术语表	知道 dsh 的能力全貌与扩展方向

验收：能给 dsh 生态贡献一个小改进或写一篇使用心得。

之后：按需深入

想做什么	去读
接外部工具	第 9 章 MCP + 官方 mcp 包文档
并行任务	第 9 章子代理
确定性流程	第 9 章工作流 + 官方 examples
生产部署	官方架构文档 + 安全模型 (sandbox/权限)
性能极致	第 6 章 + dsh-tool-turbo 源码
生态变现	第 7 章 + dsh-plugin topic

学习原则（白皮书全程）

- 1. 动手 > 阅读：每章命令都跑一遍，不看会后悔
- 2. 先复制后理解：第 4 章完整代码先跑通，再改参数理解机制
- 3. 单测 + 实机双证据：开发插件时两个都要过
- 4. 善用 --dump-config：疑惑时看合成配置，比猜快

第 1 章：认识 DeepSeek Harness

本章目标：从一个完全零基础的角度，建立对 DeepSeek Harness (dsh) 的完整认知——它是什么、为什么值得学、和主流 Agent 有什么区别、什么时候用它。读完本章，你不需要任何前置知识，就能理解 dsh 在整个 AI 开发工具版图中的位置。

TL;DR（本章核心，30 秒版）

- 1. dsh = Agent 的乐高底座：官方开源（MIT）的运行时，一切能力都是插件，你可以自由拼装
- 2. harness = 模型外面的工程层：会话、工具、上下文、循环控制——让模型在仓库里真正干活
- 3. 和 Claude Code 的区别：Claude Code 是"整车"，dsh 是"底座 + 积木"——可定制面完全不同
- 4. 生态窗口：2026-08-13 开源，中文教程此前为零——现在入场是早期
- 5. 谁该用：要深度定制/玩生态/跑 CI 的开发者；要开箱即用的选 Claude Code

1.1 先建立三个直觉

在讲技术之前，先用三个类比建立直觉：

- ① dsh 是什么？——它是"Agent 的乐高底座"。想象乐高：官方提供底板和标准积木（运行时 + 核心插件），你可以自由拼装（加插件、换界面、改行为）。对比之下，Claude Code 更像"一辆整车"——很好开，但你想改装发动机得找官方。
- ② 为什么需要"harness"这个词？——它是"套在模型外面的工程层"。一个模型（DeepSeek V4）本身只会"回复文字"。要让它在你的代码仓库里干活（读文件、跑命令、改代码、多轮循环），外面需要一层工程：会话管理、工具调用、上下文控制、错误恢复。这层工程就叫 harness。dsh 是 DeepSeek 官方把这层工程开源出来的产品。
- ③ 为什么 2026 年才开源？——因为 Agent 进入"可编程时代"。2025 年是"模型能力竞赛"（谁能生成更好的代码）；2026 年进入"Agent 工程竞赛"（谁能更好地组织模型干活）。DeepSeek 开源 harness 的战略意图：把"如何组织 Agent"这件事变成开放生态——像当年 Android 开源改变手机生态一样。

1.2 官方定义与核心事实

一句话定义：DeepSeek Harness (dsh) 是 DeepSeek 官方开源的 Agent 运行时，采用"一切皆插件" (everything is a plugin) 架构，基于 Cordis 插件容器构建。

事实	内容
开源时间	2026-08-13
协议	MIT (可商用、可改)
语言	TypeScript (Node.js ≥ 22)
版本线	0.1.0-rc.x (当前 rc.6, 迭代快, 官方明示"将有破坏性变更")
底层	Cordis (可组合插件容器)
内置形态	web (Web UI) + headless (一次性 CLI)
官方定位	官方 README 原话: "everything is a plugin"

1.3 架构是怎么"一切皆插件"的

```
flowchart TB
    subgraph Profile["你的 profile (可启动形态)"]
        P1["dsh web (Web UI)"]
        P2["dsh headless (CLI)"]
        P3["自定义 profile (TUI/桌面/机器人...)"]
    end
    subgraph Plugins["能力层 (每个能力 = 一个插件)"]
        L["llm: 模型接入 + 推理档位"]
        T["tools: 写文件/终端/搜索/技能"]
        S["session: 会话持久化"]
        C["client: 界面 (web/终端)"]
        ST["settings: 用户配置"]
    end
    subgraph Cordis["Cordis 插件容器"]
        D["依赖注入 • 事件 • 生命周期"]
    end
    Profile --> Cordis
    Cordis --> Plugins
    Plugins --> L
    Plugins --> T
    Plugins --> S
    Plugins --> C
    Plugins --> ST
```

3.1 分层视图



• 你的自定义 profile (TUI/桌面/机器人…)	
能力层 (每个能力 = 一个插件)	
• llm: 模型接入 (DeepSeek V4 系 + 推理档位)	
• tools: 工具 (写文件/终端/搜索/技能…)	
• session: 会话持久化	
• client: 界面 (web 浏览器半 / 终端半)	
• settings: 用户配置	
… (60+ 官方包)	
Cordis 插件容器: 加载、依赖注入、事件、生命周期	

3.2 三个必须懂的概念

① profile (可启动形态) 一个 profile = \$DSH_HOME/profiles/<name>/ 目录, 包含:

- package.json : 插件依赖 + 清单 (dsh.profile.bundles 指定 bundle 顺序)
- cordis.patch.yml : 你的补丁层 (挂载/覆盖插件) 启动时按顺序合成: 内置 bundle → profile patch → 全局 patch → --patch 覆盖。

② host 半 / client 半 (一个插件, 两副面孔)

半边	跑在哪	干什么
host 半	Node 进程	工具、服务、事件、文件系统——apply(ctx) 注册
client 半	浏览器 (web profile)	UI、交互——package.json 的 dsh.client 声明

一个 npm 包可同时携带两半 (exports["."] + exports["./client"]) 。

③ 扩展点 (extension point) 官方原则: "Plugins, not loop changes"——改行为优先用官方钩子, 不要 fork 核心。常用扩展点 (后续章节逐一实战) :

- agent/request waterfall: 每次模型请求前改配置 (工具调用提速插件的挂点)
- conversationEvents.register : 订阅/注入对话事件
- ctx.slots.inject : 在界面槽位注入 UI
- settings 服务: 注册用户可配置项

1.4 DSH 与主流 Agent 的全面对比

4.1 能力矩阵

维度	dsh	Claude Code	OpenAI Codex	OpenCode	Gemini CLI	Kimi CLI
开源	✅ MIT	❌ 闭源	❌ 闭源	✅ MIT	❌ 闭源	❌ 闭源
模型绑定	模型无关 (官方适配 DeepSeek)	Claude 系	GPT 系	任意	Gemini 系	Kimi 系
官方运行时	✅ (web + headless + 插件生态)	产品即运行时	产品即运行时	客户端 (无官方后端)	产品即运行时	产品即运行时
插件体系	官方级: 一切皆插件, 60+ 官方包	配置/钩子为主	配置为主	配置为主	无	无

自定义界面	✅ (client 半 = 自由 UI)	❌	❌	部分 (TUI 固定)	❌	❌
自动化/CI	✅ headless profile	✅	✅	✅	✅	✅
TUI	插件可做 (官方未内置)	✅ 内置	✅ 内置	✅ 内置	✅	✅
生态阶段	零日起步 (2026-08-13)	成熟	成熟	成熟	成熟	早期
适合谁	想深度定制 + 玩生态的开发者	开箱即用	开箱即用	熟悉 OpenCode 用户	Google 生态	Kimi 生态

4.2 案例对比：同一个任务，不同的打开方式

任务：让 Agent 在仓库里"找到所有调用某个函数的地方，并统一改一个参数"。

Agent	你会怎么做	体验
dsh (web)	dsh web → 输入指令 → 模型用 Grep/Read/Edit 工具完成	Web UI + 右侧插件侧边栏 (可加 Git 面板)
dsh (headless)	dsh --profile headless "任务" → 打印结果退出	可进 CI：非零退出码即失败
Claude Code	claude → 输入指令 → 模型完成	终端 TUI，开箱即用
Codex	codex → 输入指令	终端 TUI
OpenCode	opencode → 输入指令	终端 TUI，可配任意模型
Kimi CLI	kimi → 输入指令	终端 TUI

差异点在哪：同样一句话，dsh 让你多了一个选择维度——界面和工具链都可以换。比如把 Git 面板、工具调用提速插件挂进去，同一个模型干活的效率就不一样了。其他 Agent 的界面/工具链是官方定的。

4.3 案例对比：真实 workflow（我们的实测）

以下是我们真实开发 dsh 生态时的对比观察 (2026-08-13)：

场景	dsh 实测	备注
简单文件创建	冷启动 ~110s (首轮含上下文注入) → 热缓存 ~1s	思考档位是主要变量
50 步工具链任务	LLM 耗时 10m+, 工具调用 9m+	每步思考累计——提速插件价值在此
插件开发	从零到可运行插件：1 天 (含测试+实机验证)	扩展点清晰 (agent/request waterfall)
与 Claude Code 同任务	dsh 配 V4-Flash 成本约为 Claude 的 1/10~1/30	价格维度 dsh 生态显著占优

数据来源：本白皮书配套开发记录 (dsh-tool-turbo 实测日志 + 官方成绩单对比)。

1.5 什么时候用 dsh（选型决策）

✅ 推荐入场：

- 你要做模型无关、界面可选、行为可改的 Agent 底座
- 你想成为 dsh 生态早期贡献者（先发优势，官方点名鼓励）
- 你要在服务器/CI 跑 Agent (headless profile)
- 你对成本敏感（DeepSeek 模型 + 开源生态）

⏸ 暂时观望：

- 只要"开箱即用的编码助手"——Claude Code 等更成熟
- 不能接受 rc 阶段的破坏性变更——等 0.1.0 正式版
- 重度依赖某模型独有能力（如 Claude artifacts）——模型绑定场景

1.6 常见问题（FAQ）

Q1: dsh 是模型吗？不是。dsh 是运行时/框架，模型通过 `llm` 插件接入（官方适配 DeepSeek V4 系，理论上可接其他 OpenAI 兼容模型）。

Q2: dsh 和 OpenCode 什么关系？都是开源 Agent 客户端，但定位不同：OpenCode 是"客户端 + 配置"，dsh 是"运行时 + 官方插件生态"（有官方后端 bundle 与 60+ 官方包）。dsh 更底层、可定制面更大。

Q3: 没写过 TypeScript 能玩吗？能。使用（第 2 章）不需要编程；写插件（第 4 章）需要基础 TS，但教程给完整代码。

Q4: dsh 稳定吗？当前 rc 阶段（0.1.0-rc.6），迭代快、有破坏性变更。生产核心依赖建议等正式版；玩生态现在是时机。

Q5: 为什么现在学 dsh 值得？生态零日 + 官方点名鼓励社区 + 中文教程空白——每个早期生态都有"第一个吃螃蟹的人"的红利，现在是入场窗口。

下一章：第 2 章：五分钟快速上手 —— 装起来，跑起来。

动手练习（检验你是否真懂了）

1. 一句话测试：向不懂技术的人解释"dsh 是什么"（不能用"Agent/harness/插件"这些词）
2. 对比测试：说出 dsh 和 Claude Code 的 3 个本质区别（不是功能列表）
3. 选型测试：给下面场景选工具并说明理由：
 - 场景 A：想要开箱即用的终端编码助手
 - 场景 B：想做一个"模型无关、界面自定义"的公司内部 Agent 平台
 - 场景 C：想在 CI 里每天自动跑一个数据分析任务
4. 架构测试：画出"profile → 插件 → 扩展点"的关系图（不看原文）
5. FAQ 测试：回答"dsh 是模型吗？""没写过 TS 能玩吗？"

完成练习后，进 [第 2 章](#) 动手装起来。

第 2 章：五分钟快速上手

本章目标：跟着做，跑起来。每一条命令都给出预期输出与常见错误解法。建议打开终端边看边做。

TL;DR（本章核心，30 秒版）

1. 装： `npx -y @deepseek-ai/dsh web` → <http://127.0.0.1:3080>

- 2. 两种模式: web (对话 UI) / headless (`dsh --profile headless` “任务”, CI 友好)
- 3. 推理档位三档: low (最快) / high (默认) / max (最强) ——工具链任务 90% 时间在思考, 降档是最快提速
- 4. 模型: `deepseek-v4-flash` (默认, 性价比) 或 `deepseek-v4-pro` (旗舰)
- 5. 配置: `~/dsh/settings.yaml` (模型 + 推理档位)

2.1 准备工作 (30 秒检查)

需要	检查命令	通过标准
Node.js ≥ 22	<code>node --version</code>	v22.x 或更高 (推荐 24)
npm (随 Node 附带)	<code>npm --version</code>	有版本号即可
网络	能访问 npm registry	能装包
(可选) DeepSeek API Key	https://platform.deepseek.com	用于真实对话

没有 API Key 也能启动 dsh (界面能开), 但对话需要 Key。本白皮书示例假设已配置。

2.2 安装 (两种方式)

方式一: 直接运行 (推荐新手)

```
npx -y @deepseek-ai/dsh --version
```

首次运行会下载 dsh (包体较大, 含 40+ 插件模块, 约 1-3 分钟)。看到版本号即成功:

```
0.1.0-rc.6
```

方式二: 全局安装 (推荐频繁使用)

```
npm install -g @deepseek-ai/dsh
dsh --version
```

2.3 模式一: Web UI (`dsh web`)

启动

```
dsh web
```

预期输出:

```
dsh web: http://127.0.0.1:3080
```

浏览器打开 <http://127.0.0.1:3080>。

界面认识 (对照截图)

 dsh Web UI 对话

区域	内容
----	----

左栏	会话列表 / 工作区切换 / 新建会话
中栏	对话区：输入框、模型选择 (DeepSeek V4 Flash)、推理等级 (High)
右侧/底部	插件侧边栏 (默认空；安装社区插件后出现)
右上	Session log (会话日志) / 轨迹 (工具调用轨迹)

第一次对话

- 1. 点「新建会话」
- 2. 输入框输入： 你好，请用一句话介绍你自己
- 3. 回车发送

预期回复类似：

你好！我是 DeepSeek 驱动的 AI 编程助手，可以帮你写代码、调试问题、处理文件、搜索资料，以及完成各种开发和办公任务。

模型与推理档位

点输入框旁的「选择模型」：

模型	定位
deepseek-v4-flash (默认)	性价比：快、便宜，日常够用
deepseek-v4-pro	旗舰：更强，更贵更慢

推理等级 (思考模式三档，2026-08-13 起支持)：

档位	速度	质量	建议场景
low	最快	够用	简单/确定性任务、批量、工具链廉价轮次
high (默认)	中等	好	日常 Agent 任务
max	最慢	最强	复杂推理、长链规划

💡 性能关键认知：模型在每次工具调用前都会重新思考。实测一个“创建文件”任务，思考占 ~90% 墙钟时间；50 步工具链任务思考累计可达数分钟到十几分钟。调低推理档位是性价比最高的提速手段 (见第 6 章 + dsh-tool-turbo 插件)。

2.4 模式二：Headless (一次性任务，适合脚本/CI)

```
dsh --profile headless "你好，请用一句话介绍你自己"
```

预期输出 (打印结果后进程退出)：

你好！我是 DeepSeek 驱动的 AI 编程助手，可以帮你写代码、调试问题、处理文件、搜索资料，以及完成各种开发和办公任务。

Headless 的核心价值：

- 自动化：可进 CI、服务器、cron
- 脚本友好：非零退出码 = 失败；输出可管道处理

- 会话隔离：每次调用一个新鲜会话（`--resume` 可恢复，见 `dsh --profile headless --help`）

实战：写个脚本每天跑一次“生成日报”：

```
dsh --profile headless "读取工作区今天的 git log，生成一份中文日报摘要" > daily-report.md
echo "exit=$?"
```

2.5 你的第一个插件：给 web 加个 Git 面板

`dsh` 的侧边栏默认是空的——安装社区插件 `dsh-better-sidebar` 体验“一切皆插件”（详细原理见第 3 章，这里先跑通）：

```
# 1. 找到你的 web profile
#   Windows: %USERPROFILE%\dsh\profiles\web
#   macOS/Linux: ~/.dsh/profiles/web

# 2. 在 package.json 的 dependencies 加一行（link: 指向插件源码）
#   "dsh-better-sidebar": "link:C:\\path\\to\\DSH-better-sidebar"

# 3. 在 cordis.patch.yml 加挂载行
#   - insert:
#       - id: better-sidebar
#         name: dsh-better-sidebar

# 4. 安装并重启
cd ~/.dsh/profiles/web && pnpm install
dsh web
```

重启后，右侧出现文件管理 / 终端 / Git 面板 / 浏览器等标签：

 dsh Git 面板 (better-sidebar 插件)

图中「拉取远端 / 拉取合并 / 推送」按钮是社区 PR 实现的（见第 5 章案例）——这就是“插件生态”的运转方式。

2.6 配置与目录速查

首次运行后生成的目录：

```
~/.dsh/
├── settings.yaml      # 全局设置（模型、推理档位）
├── profiles/         # profile 目录
│   └── web/
│       ├── package.json  # 插件依赖 + 清单
│       └── cordis.patch.yml # 补丁层（挂载插件）
├── sessions/         # 会话数据
└── storages/         # 持久化存储
```

`settings.yaml` 示例：

```
agent-default-model:
  model: deepseek-v4-flash
```

reasoningEffort: high

2.7 命令速查

命令	用途
dsh web	启动 Web UI (=dsh --profile web)
dsh --profile headless "任务"	一次性任务，打印结果退出
dsh plugin --profile <name> add <pkg>	给 profile 安装插件
dsh --dump-config	打印合成配置树
dsh --profile tui	TUI 模式（需先安装 tui 插件，官方未内置）
dsh --version	版本

2.8 排障速查

现象	原因与解法
dsh: profile "tui" does not exist	tui profile 需插件创建 (dsh plugin --profile tui add <pkg>)
npx 极慢	首次下载包体大; npm i -g 后更快
浏览器打不开 3080	端口被占: netstat -ano findstr 3080 → kill PID
模型无响应	检查 ~/.dsh/settings.yaml 模型配置 + API Key
插件装不上 (404)	rc.1 依赖断裂: 确认依赖用 ^0.1.0-rc.6 线 (第 3 章常见坑 #1)
升级后行为变了	rc 阶段破坏性变更正常, 看官方 changelog

下一章: [第 3 章: profile 与插件系统](#) —— 理解可定制骨架。

动手练习 (10 分钟内完成)

- 安装: npx -y @deepseek-ai/dsh --version 确认版本
- Web 对话: 启动 dsh web, 新会话发"你好", 观察回复与界面布局
- Headless: dsh --profile headless "1+1 等于几", 确认打印结果后退出
- 推理档位实验: 把 settings.yaml 的 reasoningEffort 改为 low, 重新跑一个简单任务, 感受速度差异
- 排障演练: 模拟"端口被占" (先起一个占用 3080 的服务), 用 netstat 排查

全部通过后, 进 [第 3 章](#) 理解"为什么能这样改"。

第 3 章: profile 与插件系统

本章目标: 理解 dsh 的可定制骨架——profile 怎么组织、插件怎么挂载、host/client 双半是什么。这是从"用户"变成"开发者"的分水岭。

TL;DR (本章核心, 30 秒版)

- 1. profile = 一种可启动形态: `~/.dsh/profiles/<name>/` 目录, 由 `package.json` (插件清单) + `cordis.patch.yml` (补丁层) 组成
- 2. 挂载插件只需两步: `package.json` 加依赖 + `cordis.patch.yml` 加挂载行, 然后 `pnpm install` 重启
- 3. host 半跑在 Node, client 半跑在浏览器: 一个 npm 包通过 `exports["."]` 和 `exports["./client"]` 同时携带两副面孔
- 4. 改行为找扩展点, 别 fork 核心: `agent/request` **waterfall**、`conversationEvents`、`ctx.slots.inject`、`settings` 服务是四大常用钩子
- 5. rc 阶段三大坑: 依赖版本用 `^0.1.0-rc.6` 线、`next()` 必须 `await`、client 测试需要 dsh 运行时

3.1 profile: 一个可启动的配置栈

dsh 用 profile 表示"一种可启动的形态"。官方内置两个, 其余用插件创建:

profile	用途	命令
web	Web UI (对话 + 侧边栏 + 工具)	dsh web
headless	一次性 CLI 任务	dsh --profile headless "任务"
tui (需插件)	终端 UI	dsh --profile tui (未内置, 需安装插件)

一个 profile 目录长这样 (`~/.dsh/profiles/<name>/`) :

```
profiles/web/
├── package.json      # 插件依赖 + dsh.profile 清单 (bundles 顺序)
├── cordis.patch.yml  # 你的补丁层: 挂载/覆盖插件的声明
├── cordis.yml        # (生成的) 合成配置
├── pnpm-workspace.yml
└── node_modules/
```

加载顺序 (官方文档原文) : 内置 bundle (`dsh-base` → `dsh-web-app`) → profile 的 `cordis.patch.yml` → 用户级 `~/.dsh/cordis.patch.yml` → `--patch` 覆盖层。

3.2 挂载一个插件: 两处改动

以挂载社区插件 `dsh-better-sidebar` 为例 (真实操作) :

① `package.json` 加依赖 (`link:` 指向本地源码, 或用 npm 包名) :

```
{
  "dependencies": {
    "dsh-better-sidebar": "link:C:\\path\\to\\DSH-better-sidebar"
  }
}
```

② `cordis.patch.yml` 加挂载行:

```
- insert:
  - id: better-sidebar
    name: dsh-better-sidebar
```

③ 安装并重启:

```
cd ~/.dsh/profiles/web && pnpm install
dsh web    # 重启后生效
```

💡 插件默认是按会话隔离的 (*better-sidebar* 的布局/标签按会话持久化) ——挂载是 *profile* 级, 但状态是会话级。

3.3 host 半与 client 半：一个包，两副面孔

插件可以同时携带两个运行半：

半边	运行位置	职责	示例
host 半	Node 进程	工具、服务、事件、文件系统、进程	<code>apply(ctx)</code> 注册工具/服务
client 半	浏览器 (web profile)	UI、交互、DOM	<code>package.json</code> 的 <code>dsh.client</code> 声明 + <code>src/client/</code>

`package.json` 里如何声明 client 半：

```
{
  "dsh": {
    "client": {
      "inject": ["@deepseek-ai/dsh-client-runtime", "@deepseek-ai/dsh-client-locale"],
      "platform": "web"
    }
  },
  "exports": {
    ".": { "types": "./lib/types/index.d.ts", "default": "./lib/index.js" },
    "./client": { "types": "./lib/types/client/index.d.ts", "default":
"./lib/client.js" }
  }
}
```

- **host 半**: `exports["."] → apply(ctx)` (**cordis 插件主体**)
- **client 半**: `exports["./client"] → 浏览器侧的 apply(ctx)`
- **inject** : 声明需要的服务 (**cordis 依赖注入**)

3.4 扩展点：改行为优先找钩子，别 fork 核心

官方原则 (CONTRIBUTING/AGENTS.md 明示) : *****"Plugins, not loop changes: new behavior goes on documented extension points"*****。新手最常见的错误是改核心 `loop`——正确做法是用扩展点。

已确认的常用扩展点 (后续章节逐一实战)：

扩展点	位置	用途
<code>agent/request waterfall</code>	<code>agent-loop</code>	每次模型请求前改配置 (provider/model/reasoningEffort/tools) —— tool-turbo 用这个

agent/request-error	agent-loop	请求失败时干预（官方 compaction 插件用这个做上下文溢出恢复）
conversationEvents.register	client runtime	订阅/注入对话事件（tool/call、turn/start 等）
ctx.slots.inject	client ui-slots	在界面槽位注入 UI（如 turnTail 显示产物文件行）
settings 服务	dsh-settings	注册用户可配置的命名空间（设置页自动渲染）
ctx.provide / ctx.get	cordis	跨插件提供服务

3.5 常见坑（真实踩过）

坑	现象	解决
rc.1 依赖断裂	pnpm install 报 @deepseek-ai/dsh-type-meta@0.0.1-rc.1 404	官方 rc.1 时代多个包从未发布；升级到 ^0.1.0-rc.6 线
插件缺 main	dsh: No "exports" main defined	host 插件 package.json 要暴露入口 ("main": "src/index.ts" 可被 tsx 直接加载)
事件 handler 忘了 await next()	请求丢失 provider/model 报错	agent/request 的 next() 返回 Promise, 必须 await 后 spread
类型报 'agent/request' is not assignable to keyof Events	npm 类型未 re-export 官方类型增强	用宽松签名 (ctx.on as unknown as ...) 在边界转换
client 包产物依赖 window.__ModuleLoader__	jsdom 无法直接跑 client 测试	组件级测试需 dsh web 运行时 (官方 CI 跑)

动手练习（检验你是否真懂了）

- 1. 理解题：不看原文，画出 profiles/web/ 目录下的文件结构，并说出每个文件的作用
 自查：参考本章 3.1 节"profile 目录长这样"
- 2. 理解题：解释"host 半"和"client 半"分别跑在哪里、各负责什么。一个插件可以只有 host 半吗？
 自查：参考本章 3.3 节表格
- 3. 动手题：打开你的 ~/.dsh/profiles/web/package.json，找到 dsh.profile.bundles 字段，说出 bundle 的加载顺序
 自查：参考本章 3.1 节"加载顺序"段落
- 4. 动手题：假设你要挂载一个名为 dsh-my-widget 的插件，写出 package.json 和 cordis.patch.yml 各需要加什么内容
 自查：参考本章 3.2 节的两处改动示例

5. 动手题：在 `cordis.patch.yml` 里加一行错误的挂载（比如拼错插件名），重启 dsh web，观察报错信息并记录

自查：对比本章 3.5 节“常见坑”表格

6. 思考题：官方原则说“Plugins, not loop changes”。如果你想改 dsh 的“模型回复后自动总结”行为，应该用哪个扩展点？为什么不该直接改 agent-loop 源码？

自查：参考本章 3.4 节扩展点表格 + “改行为优先找钩子”原则

常见疑问 FAQ

Q1: profile 和 bundle 有什么区别？profile 是“一种可启动形态”（如 web、headless），bundle 是“一组预配置的插件集合”。profile 通过 `dsh.profile.bundles` 指定要加载哪些 bundle，再叠加自己的 patch。简单说：bundle 是积木包，profile 是拼好的成品。

Q2: `cordis.patch.yml` 里的 `- insert:` 是什么意思？还有其他操作吗？`insert` 是“在插件链中插入一个新插件”。patch 文件基于 cordis 的配置合并机制，支持 `insert`（插入）、`override`（覆盖已有插件配置）等操作。新手只需掌握 `insert` 即可挂载大部分插件。

Q3: 插件状态是按 profile 隔离还是会话隔离？挂载是 profile 级的（装一次，该 profile 下所有会话都能用），但状态是会话级的。比如 `better-sidebar` 的标签布局按会话持久化，不同会话互不干扰。

Q4: 我想写一个只有 client 半的插件（纯 UI），可以吗？可以。`package.json` 里声明 `dsh.client + exports["./client"]`，不写 `exports["."]` 即可。但注意：client 半无法直接访问文件系统或跑命令，需要通过 host 半的服务（`ctx.provide / ctx.get`）桥接。

Q5: `agent/request` `waterfall` 和 `conversationEvents.register` 有什么区别？我该用哪个？`agent/request` 是“改模型请求配置”的钩子（`provider/model/tools/reasoningEffort`），每次请求前触发，返回新配置。`conversationEvents.register` 是“订阅/注入对话事件”的钩子（`tool/call/turn/start` 等），用于监听或注入事件流。想改请求参数用前者，想监听对话流用后者。

Q6: 为什么我的插件装上去没反应？怎么调试？三步排查：① 确认 `pnpm install` 无报错且 `node_modules` 里有你的插件；② 确认 `cordis.patch.yml` 的 id/name 拼写正确；③ 重启 dsh web 后看进程日志有没有插件加载信息。如果 host 半有 `console.log`，日志里应该能看到输出。

下一章：第 4 章：插件开发实战（规划中）——从零写第一个 host 插件。

第 4 章：插件开发实战

本章目标：从零写一个真实可用的 host 插件——通过 `agent/request` 扩展点自动调节推理档位。这是社区项目 `dsh-tool-turbo` 的完整拆解，所有代码可运行、可测试。

TL;DR（本章核心，30 秒版）

- 核心问题：dsh 每次工具调用前模型都重新思考，50 步任务 90%+ 时间在思考——降档是最快提速
- 架构三板斧：纯函数（决策逻辑，零依赖可单测）→ 插件主体（接入 waterfall）→ 实机验证（日志证明注入发生）
- `agent/request` `waterfall`：每次模型请求前触发，监听者返回值传给下一个监听者，实现“保留原配置 + 覆盖某字段”
- `next()` 是 Promise，必须 await：不 await 直接 spread 会得到空对象，`provider/model` 丢失报错
- 开发纪律：先找扩展点（90% 行为有官方钩子）、逻辑抽纯函数、实机验证不能省

4.1 我们要做什么

问题：dsh 在每次工具调用前模型都会重新思考（`reasoning_effort`）。一个 50 步工具链任务，“思考”占 90%+ 墙钟时间。

方案：一个 host 插件，监听 `agent/request` waterfall，根据当前步骤最近的工具调用，把简单轮次的 `reasoning_effort` 从 high 降到 low。

4.2 项目骨架

```
dsh-tool-turbo/
├── package.json          # host 插件声明
├── tsconfig.json
├── src/
│   ├── effort-decision.ts # 纯函数：决策逻辑（零依赖，可单测）
│   └── index.ts           # apply(ctx)：接入扩展点
└── tests/
    └── effort-decision.spec.ts
```

package.json 关键字段：

```
{
  "name": "dsh-tool-turbo",
  "type": "module",
  "main": "src/index.ts",
  "exports": {
    ".": { "types": "./src/index.ts", "default": "./src/index.ts" }
  },
  "peerDependencies": {
    "@deepseek-ai/cordis": "^4.0.1",
    "@deepseek-ai/dsh-agent": "^0.1.0-rc.6"
  }
}
```

⚠ 依赖版本务必用 `^0.1.0-rc.6` 线——`rc.1` 线的 npm 依赖链是断的（见第 3 章常见坑）。

4.3 纯函数：决策逻辑（零依赖，可单测）

src/effort-decision.ts：

```
export type EffortId = 'low' | 'high' | 'max'

export interface ToolCallSample {
  name: string // 工具名，如 'write'、'read'、'bash'
  argsSize: number // 参数大小（字符数）
}

export interface EffortDecisionInput {
  recentCalls: readonly ToolCallSample[]
  selected: EffortId // 用户基线档
  allowDowngrade: boolean
```

```

    allowUpgrade: boolean
  }

const SIMPLE_TOOL_RE =
/^ (fs|bash|terminal|read|write|grep|glob|edit|ls|cat|rm|cp|touch|mkdir|pwd)/i
const HEAVY_ARGS = 800

export function decideEffort(input: EffortDecisionInput): EffortId {
  const { recentCalls, selected, allowDowngrade, allowUpgrade } = input
  if (recentCalls.length === 0) return selected // 全新提示：保持基线

  const ratio = recentCalls.filter(c =>
    SIMPLE_TOOL_RE.test(c.name) && c.argsSize < HEAVY_ARGS,
  ).length / recentCalls.length
  const heaviest = recentCalls.reduce((m, c) => Math.max(m, c.argsSize), 0)

  if (ratio >= 0.75 && allowDowngrade) return 'low'
  if (heaviest >= HEAVY_ARGS * 4 && allowUpgrade) return 'max'
  if (ratio < 0.75) return allowUpgrade ? 'high' : selected
  return selected
}

```

为什么拆成纯函数：决策逻辑与 dsh 运行时解耦——单元测试零依赖、毫秒级、覆盖所有分支，实机只需要验证“注入是否真的发生”。

4.4 插件主体：接入 agent/request waterfall

src/index.ts :

```

import type { Context } from '@deepseek-ai/cordis'
import { decideEffort, type ToolCallSample } from './effort-decision.ts'

export interface ToolTurboConfig {
  enabled: boolean
  allowDowngrade: boolean
  allowUpgrade: boolean
  baseline: 'low' | 'high' | 'max'
}

export const DEFAULT_CONFIG: ToolTurboConfig = {
  enabled: true, allowDowngrade: true, allowUpgrade: false, baseline: 'high',
}

const WINDOW = 8

function recentToolCalls(agent: unknown): ToolCallSample[] {
  const events = (agent as { session?: { events?: readonly unknown[] } })
    .session?.events ?? []
  const out: ToolCallSample[] = []
  for (let i = events.length - 1; i >= 0 && out.length < WINDOW; i--) {
    const e = events[i] as { type?: string; data?: { name?: string; arguments?:

```

```

unknown } } | undefined
    if (e?.type !== 'tool/call') continue
    out.push({
      name: e.data?.name ?? 'tool',
      argsSize: typeof e.data?.arguments === 'string' ? e.data.arguments.length : 0,
    })
  }
  return out.reverse()
}

export function apply(ctx: Context, config: ToolTurboConfig = DEFAULT_CONFIG): void {
  if (!config.enabled) return

  // 边界适配: npm 包未 re-export 官方事件类型增强, 这里放宽签名 (第 3 章常见坑)
  const on = ctx.on as unknown as (
    event: string,
    handler: (payload: Record<string, unknown>, next: () => unknown) => unknown |
    Promise<unknown>,
  ) => void

  on('agent/request', async (payload, next) => {
    const seed = await next() as { reasoningEffort?: unknown } // ⚠️ 必须 await!
    const calls = recentToolCalls(payload.agent)
    const effort = decideEffort({
      recentCalls: calls,
      selected: config.baseline,
      allowDowngrade: config.allowDowngrade,
      allowUpgrade: config.allowUpgrade,
    })
    console.log(`[tool-turbo] calls=${JSON.stringify(calls)} =>
    reasoningEffort=${effort}`)
    return { ...seed, reasoningEffort: effort }
  })
}

```

三个关键点 (都是真实踩过的坑) :

1. `next()` 是 **Promise**: `await next()` 拿到当前配置; 不 `await` 直接 `spread` 会得到空对象 → **provider/model 丢失** → 报错。
2. **waterfall** 语义: 监听者的返回值传给下一个监听者/最终请求。返回 `{...seed, reasoningEffort}` 就是“保留原配置 + 覆盖推理档位”。
3. `agent/request` 每步都触发: `agent-loop` 的 `buildRequest` 在每一步都会走这个 **waterfall**——所以动态决策天然按步生效。

4.5 测试

单元测试 (纯函数, 零依赖) :

```

import { describe, expect, it } from 'vitest'
import { decideEffort } from '../src/effort-decision.ts'

it('downgrades to low for simple tool chains', () => {

```

```
expect(decideEffort({
  recentCalls: [{ name: 'write', argsSize: 40 }],
  selected: 'high', allowDowngrade: true, allowUpgrade: true,
})).toBe('low')
}))
// ... 更多分支：全新提示保持基线 / 禁用降档 / 超大载荷升 max / 混合工具升 high
```

实机验证（关键——证明“注入真的发生”）：

挂载插件（第 3 章方法）→ 重启 dsh web → 发一个创建文件的任务 → 观察 dsh 进程日志：

```
[tool-turbo] agent/request: calls=[] => reasoningEffort=high
[tool-turbo] agent/request: calls=[{"name":"write",...}] => reasoningEffort=low
```

第一轮无工具调用 → 保持基线 high；检测到 write 工具 → 下一轮降为 low。注入链路完整工作。

完整可运行代码：<https://github.com/Electricitysheep/dsh-tool-turbo>

4.6 给新手的三条开发纪律

1. 先找扩展点：要改的行为 90% 有官方钩子（agent/request、settings、conversationEvents、slots）——不要 fork 核心。
2. 逻辑抽纯函数：决策/计算逻辑与 dsh 解耦 → 单测毫秒级、覆盖全分支；实机只需验证“注入发生”。
3. 实机验证不能省：单测证明逻辑，实机日志证明接线——两个都过才算完成。

动手练习（检验你是否真懂了）

1. 理解题：不看原文，说出 dsh-tool-turbo 的三层架构（纯函数层 / 插件主体层 / 测试层）各自负责什么
自查：参考本章 4.2-4.5 节的文件结构
2. 理解题：解释为什么 decideEffort 要设计成纯函数而不是直接在 apply(ctx) 里写逻辑。如果决策逻辑依赖了 ctx.session，还能单测吗？
自查：参考本章 4.3 节“为什么拆成纯函数”段落
3. 动手题：给 decideEffort 写一个新的测试用例：当 recentCalls 里有 3 个简单工具 + 1 个超大参数（argsSize = 5000）时，应该返回什么档位？写出测试代码并运行
自查：参考本章 4.5 节单元测试示例，预期结果取决于 allowUpgrade 配置
4. 动手题：在 apply(ctx) 里，如果把 await next() 改成 const seed = next()（不 await），会发生什么？写出你的推理，然后在实机中验证
自查：参考本章 4.4 节“三个关键点”第 1 条
5. 动手题：假设你要写一个类似的插件，但改为根据“当前会话的工具调用总次数”来决定档位（超过 20 次自动降为 low），写出纯函数签名和核心逻辑
自查：参考本章 4.3 节纯函数设计模式，关键是输入输出类型定义
6. 思考题：agent/request waterfall 可以有多个监听者。如果 tool-turbo 和另一个插件都监听了 agent/request，谁先执行？返回值怎么传递？
自查：参考本章 4.4 节“waterfall 语义”段落 + 第 3 章 3.4 节扩展点说明

常见疑问 FAQ

Q1: `next()` 返回的 `seed` 里到底包含什么字段？`seed` 是当前 `waterfall` 链上游累积的请求配置，通常包含 `provider`、`model`、`reasoningEffort`、`tools` 等字段。你的监听者拿到 `seed` 后，`spread` 覆盖想改的字段，其余原样传递。具体字段以官方 `agent-loop` 包的类型定义为准（`rc` 阶段可能变动）。

Q2: 为什么 `npm` 包的类型签名要“放宽”？不能直接用官方类型吗？因为 `npm` 发布的 `@deepseek-ai/dsh-agent` 等包没有 `re-export` 内部的事件类型增强（`Events` 接口没有被 `module augmentation` 扩展）。直接用 `ctx.on('agent/request', ...)` 会报类型错误。解决方案是在边界用 `as unknown as` 转换，这是 `rc` 阶段的临时方案，正式版可能修复。

Q3: 我的插件需要在每个工具调用后都触发决策，`agent/request` 够吗？够。`agent/request` 在每次模型请求前触发（包括工具调用后的下一轮请求）。你只需要在监听者里读取最近的工具调用历史（从 `payload.agent.session.events` 里倒序找 `tool/call` 事件），就能做按步决策。

Q4: 纯函数测试通过了，实机验证也通过了，但用户反馈“有时候没效果”，可能是什么原因？几种可能：① 用户的 `settings.yaml` 里 `reasoningEffort` 是 `low`，降档无效果（已经最低了）；② 用户的任务全是简单工具，本来就快，感知不到差异；③ `waterfall` 里有其他插件覆盖了你的返回值。建议加日志记录每步的输入/输出档位。

Q5: 我想给插件加一个用户可配置的开关（设置页里能开关），怎么做？用 `settings` 服务注册一个命名空间（如 `tool-turbo`），声明配置项（`enabled`、`baseline` 等）。`dsh` 的设置页会自动渲染表单，用户修改后通过 `ctx.get` 读取。具体 API 参考官方 `dsh-settings` 包文档。

Q6: `recentToolCalls` 函数里为什么要 `reverse()`？因为从 `events` 数组末尾往前遍历（取最近的 `N` 个），结果是倒序的（最新的在前）。`reverse` 后恢复时间正序（最旧的在前），和实际调用顺序一致，方便决策逻辑按“最近窗口”理解。

下一章：第 5 章：实战案例（规划中）—— Git 面板、HTML 草稿预览、提速插件。

第 5 章：dsh 能帮你做什么——应用场景全景

本章目标：抛开“如何开发 `dsh`”的视角，站在使用者角度，看 `dsh` 能在哪些日常场景真正帮到你。每个场景都有可复制的任务示例与真实耗时参考。

TL;DR（本章核心，30 秒版）

- 1. `dsh` 是通用 Agent 运行时：不只是编码助手——数据分析、文档生成、自动化、代码理解都能干
- 2. 五个核心场景：日常编码 / 数据处理 / 文档知识 / 自动化运维 / 代码理解与重构
- 3. 用法一致：一句话描述目标 + 写清楚验收标准，`dsh` 会自动编排工具链完成
- 4. 真实速度：简单任务秒级、复杂多步任务 1-5 分钟（V4-Flash + high 档实测）
- 5. 提示词黄金法则：告诉 `dsh`“完成的标准是什么”（如“运行验证通过”），比描述过程更有效

5.1 场景总览

场景	典型任务	dsh 的工具	实测参考
日常编码	写函数、修 bug、补测试	write / read / bash / grep	5-bug 修复+49 测试：94s
数据处理	清洗、分析、可视化、报告	read / write / bash / python	数据清洗+可视化：186s
文档知识	API 文档、教程、总结、翻译	read / write	API 文档生成：1m04s
自动化运维	批量处理、CI 任务、日志分析	bash / headless	headless 脚本化
代码理解	重构、审查、技术调研	grep / glob / read / bash	代码重构：4m37s

所有实测来自白皮书第 10 章真实案例（合成数据），详见 [benchmark](#)。

5.1.1 高缓存命中率：dsh 的成本秘密武器

这是 dsh 生态最被低估的优势。DeepSeek API 的 context cache（提示词缓存）：重复/相似的输入 token 按缓存价计费。实测（真实 dsh 会话统计）**缓存命中率可达 97%**。

为什么 dsh 缓存命中率特别高：

原因	说明
会话延续	同会话多轮对话，系统提示/技能目录/历史重复 → 命中
稳定的工具 schema	每步请求携带相同的工具定义 → 命中
Agent 工作负载特性	多步工具链反复携带相同上下文 → 命中率天然高

成本影响（98% 缓存折扣）：

计费项	未命中价	缓存命中价	折扣
输入（Flash）	\$0.14/M	\$0.0028/M	98%
输入（Pro）	\$0.435/M	\$0.003625/M	99%+

实际意义：一个长会话 Agent 任务，绝大部分输入 token 走缓存价——实测中“50 步工具链任务、2.4M 输入 token”的成本远低于按未命中价计算的预期。对高频/批量/长会话场景，这是数量级的成本优势。

怎么让缓存命中率更高：保持会话延续（不要频繁新开会话）、稳定 prompt 前缀（系统提示/技能目录不要反复变化）、批量任务放同一会话/同前缀。

5.2 场景一：日常编码（最适合入门的场景）

典型任务：

- 按需求写一个新函数/模块
- 修复 bug（定位 → 修改 → 验证）
- 补单元测试

任务示例（headless 一键跑）：

```
dsh --profile headless "审查 buggy_calculator.py: 找出所有 bug 修复，写完整测试，运行验证全部通过"
# 实测：5 个 bug 全修复 + 49 个测试通过（94s）
```

为什么 dsh 适合：编码是工具链最完整的场景——读文件、写代码、跑命令、看结果，闭环顺畅。产物（改的文件）会在对话末尾直接可打开。

提示词技巧：一定要写“运行验证”——dsh 会照着闭环；只写“帮我修 bug”它会修完就停。

5.3 场景二：数据处理与分析

典型任务：

- 数据质量检查与清洗
- 统计分析
- 可视化（图表）

- 生成分析报告

任务示例：

```
dsh --profile headless "分析 sales_data.json 的数据质量（缺失/类型错/重复/异常），写清洗脚本+可视化脚本，运行验证，总结处理方案"  
# 实测：52→35 行归零 + 图表 + 权衡说明（186s）
```

亮点（实测）：dsh 不仅执行，还会说明权衡——“中位数填充会产生尖峰”“异常值是否该删取决于业务”——这是 agent 工程层“会思考”的直接证据。

5.4 场景三：文档与知识工作

典型任务：

- 从代码生成 API 文档
- 把笔记整理成结构化文档
- 总结长文、翻译

任务示例：

```
dsh --profile headless "为 user_module.py（8 个函数）生成完整 Markdown API 文档：参数表、返回值、异常、示例、边界情况"  
# 实测：423 行 ReadTheDocs 风格文档（1m04s），主动识别“文档契约 vs 代码实现”差异
```

亮点（实测）：dsh 会防御式写作——标注调用陷阱（负数 limit 行为、空参数行为）、发现文档声明与实现不符的地方如实说明。

5.5 场景四：自动化与运维（headless 是王牌）

典型任务：

- 每天定时跑数据任务（日报、监控摘要）
- 批量文件处理（重命名、格式转换）
- CI 里的自动化检查

任务示例（脚本化）：

```
# cron 每天 8 点生成日报  
dsh --profile headless "读取工作区 git log，生成中文日报" > daily-report.md  
  
# 批处理：目录里所有 .txt 转 .md  
dsh --profile headless "把 docs/ 下所有 .txt 转成 .md（保留结构）"
```

为什么 headless 是王牌：进程级工具（bash/文件/搜索）全可用 + 非零退出码即失败 + 输出可管道——**它是“能写进 CI 的 Agent”**。

5.6 场景五：代码理解与重构

典型任务：

- 把过程式代码重构为 OOP
- 全仓库代码审查
- 技术调研（“这个模块怎么工作的”）

任务示例：

```
dsh --profile headless "把 legacy_orders.py 重构为面向对象+职责分离，保持行为一致，写测试验证重构前后输出相同"  
# 实测：17/17 测试通过，含脚本级产物对比（4m37s）
```

亮点（实测）：dsh 的工程意识——识别重复代码合并单一入口、保持向后兼容、发现沙箱下 tempfile 权限问题主动换方案。

5.7 让场景落地的三个提示词法则

- 1. 写验收标准，不写过程：“运行验证通过” > “写一个脚本”——dsh 自己会编排步骤
- 2. 给上下文：一句话说明清背景（文件在哪、数据长什么样、目标读者是谁）
- 3. 一次一个任务：复杂目标拆成多轮（每轮一个小闭环），比一次给巨型任务更可靠

5.8 我的第一个真实任务（15 分钟上手）

- 1. 建一个测试目录，放一个你手头的小文件（代码/数据/笔记）
- 2. 跑：dsh --profile headless “总结这个文件，指出 3 个可改进点”
- 3. 跑：dsh --profile headless “给这个文件写一个测试/生成文档”
- 4. 对比输出，感受 dsh 的“完成度”——它是不是只给了个大概？验收标准写清楚了吗？

动手练习

- 1. 场景识别：给你手头的 3 个日常任务，判断各属于哪个场景（编码/数据/文档/自动化/理解）
- 2. 提示词对比：对同一任务写两个版本提示词——一个“描述过程”、一个“写验收标准”，跑一遍对比结果
- 3. headless 脚本化：把“每日 git 日报”写成一行 cron/bash 命令，验证输出
- 4. 多步任务：设计一个跨场景任务（如“读代码 → 生成文档 → 输出报告”），看 dsh 如何编排
- 5. 思考题：什么场景 dsh 不适合？（提示：实时交互、需要人工审批的敏感操作）

常见疑问（FAQ）

- Q: dsh 只是编码助手吗？不是——工具链（读/写/跑/搜）让它通用，数据处理/文档/自动化都验证过
- Q: 复杂任务会跑多久？实测 1-5 分钟（V4-Flash）；要更快可以降推理档（第 6 章）
- Q: 任务失败怎么办？看输出里的工具调用轨迹定位；多数是“验收标准不清晰”，重写提示词重跑
- Q: 能自动化运行吗？能——headless + 非零退出码 + 管道输出，可进 cron/CI
- Q: 安全吗？工具执行有沙箱与审批层；敏感操作建议人工确认（第 8 章安全模型）
- Q: 和直接用 Claude Code 有什么区别？同模型下能力接近，但 dsh 可自定义工具链/界面/插件（第 1 章矩阵）

下一章：[第 6 章：进阶与性能调优](#)

5.9 行业视角：dsh 在不同领域的打开方式

同一套工具链，不同行业的用法。以下为通用场景（不含真实业务数据），合规敏感领域（医疗/法律）请务必人工审核。

金融与投研

- 任务：财报摘要、研报要点提取、因子说明文档、投资备忘
- 示例：dsh --profile headless “读取这份财务数据，生成含同比/环比变化的摘要报告”
- dsh 优势：数据处理管线顺手 + 高缓存命中（长报告会话便宜）

教育与学习

- 任务：教案生成、题目解析、错题归纳、学习计划
- 示例：`dsh --profile headless` “把这章知识点整理成 10 道自测题（含解析）”
- dsh 优势：文档生成 + 结构化为强项，可批量出题/归纳

销售与客服

- 任务：FAQ 整理、话术生成、客户反馈归类
- 示例：`dsh --profile headless` “把 100 条客户反馈按问题类型归类并给出改进建议”
- dsh 优势：批量文本处理 + 分类归纳

运营与内容

- 任务：周报月报、数据看板说明、内容选题、SEO 摘要
- 示例：`dsh --profile headless` “读取本周数据文件，生成运营周报（含图表代码）”
- dsh 优势：数据 + 文档 + 自动化的组合

研究与综述（学术/技术）

- 任务：论文要点提取、技术调研、文献对比
- 示例：`dsh --profile headless` “对比这 3 篇论文的方法与结论，输出对比表”
- dsh 优势：长文档处理 + 结构化输出（1M 上下文）

⚠️ 合规提醒

医疗诊断、法律意见、财务建议等高风险场景：dsh 输出需人工审核，不能直接用于决策——工具执行有沙箱，但内容责任在使用者。

第 6 章：进阶与性能调优

本章目标：从“能跑”到“跑得好”——推理档位策略、工具调用耗时分析、真实踩坑清单。

TL;DR（本章核心，30 秒版）

1. 时间花在哪：模型思考占 ~90%，工具执行 <1%，网络/渲染 ~10%——优化思考时间是性价比最高的提速
2. 三档策略：`low`（简单/批量轮次）、`high`（日常默认）、`max`（复杂推理/debug）——手动 UI 选或插件自动调
3. 耗时可视化：Web UI 底部统计行（LLM Xs / 工具调用 Ys）是最快的瓶颈定位手段
4. 7 个真实坑：`rc.1` 依赖断裂、插件缺 `main`、`next()` 忘 `await`、事件类型不识别、`client` 测试跑不了、缓存命中误判、端口占用
5. 评测三问：谁测的、什么 harness、验证器多严——跑分要带条件看

6.1 性能模型：dsh 的时间花在哪

实测一个“创建文件”任务的耗时分布：

阶段	占比	说明
模型思考 (Think)	~90%	每次工具调用前都会重新思考，是绝对大头
工具执行	<1%	文件写入等毫秒级
网络/渲染	~10%	API 往返 + UI 更新

推论：

- 简单任务 → 优化思考时间（降推理档）
- 长工具链任务 → 每步都省思考时间，累计收益最大

- 工具本身慢（搜索/大文件）→ 优化工具实现，不是调档

6.2 reasoning_effort 策略（官方三档）

档位	场景建议
low	简单/确定性轮次：文件操作、批量、工具链中的廉价步
high	日常 Agent 任务（默认）
max	复杂推理、长链规划、debug

手动：UI 的"推理等级"选择器，或 `~/.dsh/settings.yaml` 的 `reasoningEffort` 。自动：插件按工具轮次动态降档（`dsh-tool-turbo`，第 4 章）。

6.3 工具调用耗时可视化

dsh 的会话统计行（Web UI 底部）显示：`N 轮 • M 步 | LLM Xs • 工具调用 Ys | 首 token 平均 ...`——这是最快的瓶颈定位手段。

进阶：host 插件监听工具事件做 per-tool 计时（`tool-turbo` 已内置 host 日志版本），把"哪个工具最慢"暴露出来。

6.4 常见坑清单（真实踩过，含解法）

#	坑	现象	解法
1	rc.1 依赖断裂	<code>pnpm install</code> 404 (<code>dsh-type-meta</code> 等从未发布)	依赖用 <code>^0.1.0-rc.6</code> 线
2	插件缺 main	No "exports" main defined	暴露 . 入口; <code>"main": "src/index.ts"</code> 可被 <code>tsx</code> 加载
3	<code>next()</code> 忘 <code>await</code>	<code>provider/model</code> 丢失报错	<code>agent/request</code> 的 <code>next()</code> 返回 <code>Promise</code> ，必须 <code>await</code>
4	事件类型不识别	<code>'agent/request'</code> is not assignable to keyof Events	npm 未 re-export 类型增强，边界放宽签名
5	client 测试跑不了	<code>jsdom</code> 报 <code>window.__ModuleLoader__ undefined</code>	client 产物依赖 dsh 引导机制，组件测试在官方 CI 跑
6	简单任务"突然变快"的误判	1s vs 110s 差异被误归因	DeepSeek context cache 命中也会提速——A/B 测试要用全新 prompt
7	端口占用	<code>dsh web</code> 起不来	<code>`netstat -ano</code>

6.5 评测视角：官方成绩单 vs 独立实测

（结合 0813 正式版发布）看 Agent 模型成绩单要三问：

1. 谁测的？官方自测（自家 Harness）vs 独立第三方（AA 等）
2. 什么 harness？框架不同分数差很大（官方 Terminal-Bench 87.9 vs AA 独立 79）
3. 验证器多严？宽松验证器（SWE-bench Verified 8.5% 假阳性）vs 严格（DeepSWE 0.3%）

dsh 的 `agent/request waterfall` 让模型跑分可复现——这是它相比闭源产品的工程优势。

动手练习（检验你是否真懂了）

1. 理解题：不看原文，说出 dsh 任务耗时的三个组成部分及各自占比。为什么“优化思考时间”是性价比最高的提速手段？

自查：参考本章 6.1 节耗时分布表格

2. 理解题：解释 low / high / max 三个推理档位分别适合什么场景。如果一个任务“需要读 10 个文件然后做简单汇总”，应该用哪个档位？

自查：参考本章 6.2 节三档策略表格

3. 动手题：打开 dsh web，跑一个“创建 3 个文件”的任务，观察 Web UI 底部的统计行（LLM Xs / 工具调用 Ys），记录数据并分析瓶颈在哪

自查：参考本章 6.3 节“耗时可视化”段落

4. 动手题：把 ~/.dsh/settings.yaml 的 reasoningEffort 从 high 改为 low，跑同一个简单任务，对比耗时差异。再改回 high，跑一个复杂推理任务，对比差异

自查：参考本章 6.2 节“手动”段落

5. 动手题：模拟“端口占用”坑（先起一个服务占 3080 端口），用 netstat -ano | findstr 3080 找到 PID 并 kill，然后正常启动 dsh web

自查：参考本章 6.4 节坑 #7

6. 思考题：本章 6.5 节说“评测要三问：谁测的、什么 harness、验证器多严”。请用一个具体例子解释：同一个模型，在不同 harness 下跑分为什么会差很多？

自查：参考本章 6.5 节“官方 Terminal-Bench 87.9 vs AA 独立 79”的例子

常见疑问 FAQ

Q1：为什么我的 dsh “有时候特别快，有时候特别慢”？两个主要原因：① DeepSeek context cache 命中时，首轮上下文注入会快很多（1s vs 110s 的差异可能来自这里）；② 推理档位不同，思考时间差几倍。做 A/B 对比测试时，要用全新 prompt（避免缓存命中）+ 固定档位。

Q2：dsh-tool-turbo 真的能提速多少？有没有实测数据？提速收益取决于任务类型。简单任务（1-3 步）dsh 本来就快，插件效果不明显。长工具链任务（20-50 步）收益最大，因为每步思考降档的累计效果显著。具体数据参考 tool-turbo 仓库的实测日志。

Q3：我把推理档位设为 low，模型质量会下降很多吗？取决于任务。简单文件操作、批量处理、确定性任务用 low 质量几乎无差别。复杂推理、长链规划、debug 用 low 可能漏掉关键步骤。建议：日常用 high，批量/简单任务手动切 low 或装 tool-turbo 自动降档。

Q4：坑 #6 说的“context cache 命中”是什么？怎么判断是否命中？DeepSeek API 会对重复/相似的上下文做缓存，命中时首 token 时间大幅缩短。判断方法：看 dsh 会话日志里的“首 token 平均”指标，如果某次突然从 10s+ 降到 1s 以下，大概率命中了缓存。做性能对比测试时，用全新 prompt 避免缓存干扰。

Q5：rc 阶段的破坏性变更会影响性能吗？升级后需要注意什么？可能影响。rc 阶段的 agent-loop、waterfall 签名、插件 API 都可能变。升级后：① 跑一遍 dsh --version 确认版本；② 看官方 changelog 有没有 breaking changes；③ 跑一个你熟悉的任务对比耗时和行为；④ 自定义插件检查类型兼容性。

Q6：我想做性能基准测试（benchmark），有什么建议？三个原则：① 固定环境（同模型、同网络、同档位）；② 用全新 prompt 避免缓存；③ 多次运行取中位数（单次波动大）。记录：墙钟时间、LLM 耗时、工具调用耗时、首 token 时间。参考本章 6.1 节的耗时分布模型设计你的 benchmark。

下一章：第 7 章：生态与资源（规划中）。

第 7 章：生态与资源

本章目标：给你一份“加入 dsh 生态”的地图——官方入口、社区现状、以及参与方式。

TL;DR (本章核心, 30 秒版)

- 1. 官方入口：GitHub 仓库（源码 + issue）、API 文档、Discord、Discussions——当前贡献入口以 Discussions 为主
- 2. 外部 PR 暂不接受（2026-08-13 时点），但官方鼓励做 dsh-plugin 生态项目、写教程/博客
- 3. 插件生态快照：DSH-better-sidebar（最完整社区插件）、dsh-tool-turbo（提速插件）、本白皮书——发现插件搜 `topic:dsh-plugin`
- 4. 新手参与路径：用起来 → 小改进（给社区插件提 PR）→ 发插件 → 写内容
- 5. 生态零日 = 先发优势：每个早期生态都有“第一个吃螃蟹的人”的红利，现在入场正是时候

7.1 官方入口

资源	地址	用途
官方仓库	https://github.com/deepseek-ai/deepseek-harness	源码、架构文档、issue
API 文档	https://api-docs.deepseek.com	模型、定价、API 指南
Discord	官方 README 内链接	社区讨论
Discussions	官方仓库 Discussions	提案/求助（官方当前建议的贡献入口）

注意：官方 CONTRIBUTING 明确“目前不接受外部 PR”（2026-08-13 时点）——但鼓励：

- 在 Discussions 提建议（官方会评估）
- 做 dsh-plugin 生态项目（官方点名认可的方式）
- 写教程/博客

7.2 插件生态 (2026-08-13 快照)

项目	定位	状态
DSH-better-sidebar	文件管理/终端/Git/浏览器侧边栏	社区最完整插件
dsh-tool-turbo	工具调用提速（reasoning_effort 自动调节）	社区提速插件
dsh-handbook（本白皮书）	新手教程	生态文档

发现插件：GitHub 搜 `topic:dsh-plugin`。发布插件：给你的仓库加 `dsh-plugin` `topic` + `npm` 发布。

7.3 如何参与生态 (新手路径)

- 1. 用起来：`dsh web` + 装两个社区插件，跑通日常
- 2. 小改进：给社区插件提 PR（读第 5 章的三个案例，那是完整的 PR 范式）
- 3. 发插件：从第 4 章的最小 host 插件起步，挂 `dsh-plugin` `topic`
- 4. 写内容：教程/测评/避坑文（官方鼓励），与本白皮书互相引用

7.4 推荐阅读路径

目标	路径
快速上手	第 2 章 → 装 better-sidebar → 日常用
开发插件	第 3 章 → 第 4 章 → 抄第 5 章案例
性能调优	第 6 章 → tool-turbo 源码
深度定制	官方 AGENTS.md (架构) → docs/architecture.md → packages/ 源码

结语

dsh 是 2026-08-13 才开源的项目——**生态的每一天都是"早期"**。白皮书会随 dsh 演进持续更新。如果某章命令失效，大概率是 rc 版本迭代所致——以官方 changelog 为准。

祝你在这个全新的生态里，抢到自己的位置。🚀

动手练习（检验你是否真懂了）

- 1. 理解题：不看原文，说出 dsh 生态的 3 个官方入口（仓库/API 文档/社区）各自的用途
 自查：参考本章 7.1 节官方入口表格
- 2. 理解题：官方当前"不接受外部 PR"，但鼓励哪三种参与方式？为什么"做 dsh-plugin 生态项目"是官方点名认可的方式？
 自查：参考本章 7.1 节"但鼓励"段落
- 3. 动手题：在 GitHub 搜索 `topic:dsh-plugin`，列出你找到的 3 个社区插件，说出每个插件的定位
 自查：参考本章 7.2 节"发现插件"段落
- 4. 动手题：按本章 7.3 节的"新手路径"，完成第一步（用起来）：装 `dsh web` + 装 `better-sidebar` 插件 + 跑通一个日常任务，记录你的体验
 自查：参考本章 7.3 节第 1 步 + 第 2 章 2.5 节插件挂载教程
- 5. 动手题：给 `DSH-better-sidebar` 或 `dsh-tool-turbo` 的 README 提一个改进 PR（比如加一个安装步骤、修一个 typo），体验社区 PR 流程
 自查：参考本章 7.3 节第 2 步 + 第 5 章案例的 PR 范式
- 6. 思考题：本章结语说"生态的每一天都是早期"。如果你现在入场做 dsh 生态，你会选择做什么方向的插件？为什么？
 自查：参考本章 7.2 节插件生态快照 + 7.4 节推荐阅读路径

常见疑问 FAQ

- Q1：官方说"不接受外部 PR"，那我的插件代码放哪？放在你自己的 GitHub 仓库，作为独立的 dsh-plugin 生态项目。官方鼓励这种方式——你不需要改 dsh 核心代码，而是通过插件扩展能力。给你的仓库加 `dsh-plugin` `topic`，npm 发布即可。
- Q2：Discord 和 Discussions 有什么区别？我该用哪个？Discord 适合实时讨论、快速问答；Discussions 适合正式提案、功能建议、求助（有记录可查）。官方当前建议的贡献入口是 Discussions——如果你想让官方评估你的建议，在 Discussions 提。
- Q3：我想写中文教程/博客，有格式要求吗？没有官方格式要求。建议：包含安装步骤、代码示例、实机截图/日志。可以参考本白皮书的结构（TL;DR + 分步讲解 + 练习）。写完后可以在 Discussions 或 Discord 分享链接。

Q4: 生态零日是什么意思？为什么说是"先发优势"？生态零日 = 生态刚起步，内容/插件/教程几乎为零。先发优势 = 早期入场者容易成为"某个方向的标杆"（比如第一个做 TUI 插件的人、第一个写中文教程的人）。类比：2015 年写 React 教程的人、2018 年做 VS Code 插件的人。

Q5: rc 阶段迭代快，我的插件会不会刚写完就过时了？有可能。rc 阶段的插件 API 可能有破坏性变更。降低风险的方法：① 依赖用 `^0.1.0-rc.6` 线（当前最新）；② 关注官方 changelog；③ 插件逻辑尽量用纯函数隔离，接入层（waterfall/事件）薄一点，升级时只改接入层。

Q6: 我想做 dsh 生态项目，但不知道从哪开始，有什么建议？从"自己的痛点"开始。你用 dsh 时觉得哪里不方便？那个不方便就是一个插件机会。比如：① 想要 TUI 模式 → 做 `tui profile` 插件；② 想要 token 成本追踪 → 做 `cost-tracker` 插件；③ 想要更好的 diff 视图 → 做 `diff` 插件。参考本章 7.4 节推荐阅读路径，按目标选路径。

第 8 章：工具与上下文系统

本章目标：理解 dsh 的"能力引擎"——模型能调用哪些工具、上下文是怎么喂给模型的、以及长对话怎么处理。这是从"能跑"到"跑得明白"的关键一章。

TL;DR（本章核心，30 秒版）

- 1. 60+ 官方能力包：工具（`fs/shell/web/skill/todo`）、上下文（`context/compaction`）、会话、子代理、MCP、工作流、安全、模型、界面——一切皆插件
- 2. 内置工具名是简短动词：`read / write / grep / glob / edit / bash / todo / skill` ——写提示词直接说"读文件""搜索"即可
- 3. 工具返回 `locations` → 产物追踪：模型改了什么文件，UI 能直接看到并可打开（对话末尾的产物 `chips`）
- 4. 上下文 = 系统提示 + 技能目录 + 对话历史 + 工具结果：分层注入，每步请求携带工具 `schema`
- 5. 长对话自动压缩（`compaction`）：检测溢出 → 修剪历史 → 可选摘要 → 失败路由到溢出代理。重要信息建议写进提示词

8.1 官方能力包地图（60+ 包一览）

dsh 的能力全部以包形式提供（`packages/<group>/<name>`）。新手最需要认识的：

能力域	官方包	作用
工具 (tools)	<code>fs/tool-fs</code> 、 <code>fs/tool-fs-search</code> 、 <code>fs/tool-str-replace-editor</code> 、 <code>shell/tool-bash</code> 、 <code>web</code> 、 <code>skill</code> 、 <code>todo</code>	文件/终端/网页/技能/待办等可调用工具
上下文 (context)	<code>context/*</code> 、 <code>compaction/*</code>	请求上下文组装、长对话压缩
会话 (session)	<code>session/*</code>	持久化、标题、遥测
子代理 (subagent)	<code>subagent/*</code>	委派子任务
MCP	<code>mcp/*</code>	MCP 客户端（外部工具服务器）
工作流 (workflow)	<code>workflow/*</code>	多步工作流编排
安全 (safety)	<code>sandbox/*</code> 、 <code>guard/*</code> 、 <code>interaction/*</code>	沙箱、循环卫生、权限/审批

模型 (llm)	llm/*、llm-deepseek、llm-retry	模型接入、重试
技能 (skill)	skill/*	技能提供者注册表
界面 (client)	client/* (ui-conversation、ui-tool...)	Web UI 各部件

完整清单见官方仓库 `packages/AGENTS.md`。

8.2 内置工具（实测观察）

模型实际可调用的工具名是简短动词（实测记录，来自 dsh web 会话与 agent/request 日志）：

工具名	作用	备注
read	读文件	fs 能力
write	写文件	fs 能力
grep	内容搜索	fs-search
glob	文件模式匹配	fs-search
edit / str_replace_editor	精准编辑	工具结果含 locations（用于产物追踪）
bash / pwsh	执行命令	沙箱隔离
todo	待办管理	长任务规划
skill	技能调用	skill-catalog 注入

工具结果与产物追踪（重要概念）：工具的返回里带有 `locations`（文件路径），dsh 用它们做“产物文件行”（对话结尾的产物 chips 就是从这里来的）——模型改了什么文件，UI 能直接看到并可打开。

8.3 上下文是怎么喂给模型的

一次模型请求的上下文 = 系统提示 + 技能目录 + 对话历史 + 工具结果。实测在会话日志中可见：

上下文注入 @deepseek-ai/dsh-system-prompt ← 官方系统提示

上下文注入 skill-catalog ← 技能目录

dsh 的上下文机制：

- 系统提示分层：官方插件通过 `systemPrompt.section()` 注册提示片段（如 `ui-deliverables` 注册“产物文件引用”指导）
- 技能目录注入：可用技能列表进入上下文，模型按需调用
- 工具 schema：每步请求携带工具定义

8.4 长对话：compaction（压缩）

长对话会撑爆上下文。dsh 的 `compaction` 插件（如 `compaction-basic`）负责：

- 检测上下文溢出（`agent/request-error` 的 `CONTEXT_WINDOW_EXCEEDED`）
- 压缩历史（模型无关的修剪 + 可选摘要）
- 失败时路由到“溢出代理”（`overflow agent`）

对新手：知道“长对话会自动压缩”即可，细节是进阶话题。生产环境注意：压缩会丢细节，重要上下文建议手动写进提示词。

8.5 权限与安全模型（了解即可）

- 访问模式：UI 里可见「Workspace Write」等模式（权限预设）
- 交互审批：`interaction/*` 提供权限/审批能力（危险操作可要求确认）
- 沙箱：`sandbox/*` 隔离命令执行（如 pwsh 沙箱有 ACL 约束——实测中遇到过 temp 目录权限问题）
- 工具超时：`guard/*` 提供 loop 卫生与工具超时

安全配置的深度话题超出本白皮书范围；核心认知：*dsh* 的工具执行默认有隔离与审批层，不是裸执行。

8.6 新手最该记住的三件事

1. 工具名是简短动词（read/write/grep/glob/edit/bash）——写提示词/插件时直接说“读文件”“搜索”即可
2. 工具返回 locations → 产物追踪——模型改的文件会出现在对话产物区
3. 长对话自动压缩——不必手动清理历史（但重要信息要写进提示词）

动手练习（检验你是否真懂了）

1. 理解题：不看原文，列出 *dsh* 内置的 8 个工具名（简短动词）及各自作用

自查：参考本章 8.2 节工具表格

2. 理解题：解释“工具返回 locations → 产物追踪”是什么意思。模型改了什么文件，UI 怎么知道？

自查：参考本章 8.2 节“工具结果与产物追踪”段落

3. 动手题：在 *dsh web* 里发一个“创建一个 hello.py 文件”的任务，观察对话末尾的“产物 chips”，点击打开文件，验证产物追踪是否工作

自查：参考本章 8.2 节“产物文件行”说明

4. 动手题：在 *dsh web* 里进行一个长对话（20+ 轮），观察是否触发 compaction（看会话日志有没有“上下文溢出”或“压缩”相关日志）

自查：参考本章 8.4 节“长对话自动压缩”段落

5. 思考题：为什么“重要信息要写进提示词”而不是依赖对话历史？compaction 压缩时会丢什么？

自查：参考本章 8.4 节“压缩会丢细节”警告

6. 思考题：本章 8.5 节说“*dsh* 的工具执行默认有隔离与审批层”。如果你要做一个“允许模型自动执行所有 bash 命令”的插件，应该用哪个扩展点？有什么安全风险？

自查：参考本章 8.5 节“交互审批”+ `interaction/*` 包说明

常见疑问 FAQ

Q1：工具名是 `read / write` 这些简短动词，我写提示词时也要用这些名字吗？不需要。你写“读一下这个文件”“搜索包含 foo 的行”，模型会自动映射到对应工具。简短动词名是插件内部实现细节，提示词用自然语言即可。

Q2：`edit` 和 `str_replace_editor` 是同一个工具吗？是。`str_replace_editor` 是内部包名，模型调用时可能显示为 `edit`。功能一样：精准替换文件中的某段文本。工具结果里的 `locations` 用于产物追踪。

Q3：`compaction` 压缩后，模型还能记得之前对话的内容吗？记得“摘要”，但会丢细节。`compaction` 会修剪历史 + 可选生成摘要，关键信息保留，细节丢失。所以重要上下文（如项目约束、特殊要求）建议写进系统提示或提示词，不要只依赖对话历史。

Q4：`context/*` 和 `compaction/*` 有什么区别？`context/*` 负责“组装每次请求的上下文”（系统提示 + 技能目录 + 对话历史 + 工具 schema）。`compaction/*` 负责“长对话压缩”（检测溢出 → 修剪 → 摘要）。一个是“装什么”，一个是“装不下时怎么压缩”。

Q5: 安全模型里"沙箱"和"交互审批"有什么区别? 沙箱 (`sandbox/*`) 隔离命令执行环境 (如 `pwsh` 沙箱有 ACL 约束, 限制文件访问)。交互审批 (`interaction/*`) 是"危险操作前要求用户确认" (如删除文件、执行未知命令)。一个管"在哪执行", 一个管"能不能执行"。

Q6: 我想给 `dsh` 加一个新工具 (比如"查数据库"), 应该怎么做? 两种方式: ① 写一个 `host` 插件, 用 `ctx.provide` 注册工具服务 (参考第 3 章 3.4 节扩展点); ② 用 MCP 接入外部工具服务器 (第 9 章 9.1 节)。前者适合深度集成, 后者适合快速接入现有系统。

下一章: [第 9 章: MCP、子代理与 workflow](#)

第 9 章: MCP、子代理与 workflow

本章目标: 了解 `dsh` 的三项扩展能力——MCP (接入外部工具)、子代理 (并行任务)、workflow (多步编排)。这些是把 `dsh` 从"单 Agent"升级为"Agent 系统"的开关。

⚠ 本章基于官方架构文档 (`packages/AGENTS.md`) 与包结构整理; 部分功能示例标注「待实测」——`rc` 版本迭代快, 具体用法以官方 `changelog` 为准。

TL;DR (本章核心, 30 秒版)

1. MCP = 给 Agent 插外部工具: 通过 MCP 协议接入数据库、浏览器、公司内部系统等——先跑通内置工具, 有明确缺口再加 MCP
2. 子代理 = 并行干活: 把任务委派给子 Agent 并行执行 (大仓调研、长任务分解、独立验证) ——先用好单 Agent 再上子代理
3. workflow = 确定性流程编排: 和"多轮对话"不同, workflow 把步骤定义成流程按顺序/条件执行——适合数据拉取 → 清洗 → 报表 → 校验
4. 组合起来 = Agent 系统: 父 Agent 规划 → 子代理并行调研 → 工具链 + MCP → workflow 汇总 → 产物
5. 新手路径四阶段: 单 Agent + 内置工具 → + MCP → + 子代理 → + workflow (逐步叠加, 别跳级)

9.1 MCP: 接入外部工具生态

MCP (Model Context Protocol) 是"给 Agent 插外部工具"的开放协议。`dsh` 提供 `mcp` 客户端包 (`@deepseek-ai/dsh-mcp-client`)。

能做什么: 通过 MCP 接入任何 MCP 服务器 (数据库、浏览器、图表、公司内部系统.....) ——比如社区已出现的 `dsh-plugin-cost-tracker` (追踪 token) 就是 MCP/插件形态。

新手定位: MCP 是"生态扩展", 等你在 `dsh` 里有了明确的工具缺口再来接。先跑通内置工具, 再加 MCP。

9.2 子代理 (subagent) : 并行干活

是什么: 把任务委派给子 Agent 并行执行 (`packages/subagent/*`)。

典型场景:

- 大仓库多模块调研 (各模块一个子代理)
- 长任务分解 (父代理规划, 子代理执行)
- 独立验证 (子代理交叉检查)

对新手: 子代理是"高阶武器"——先用好单 Agent, 理解任务分解后再上子代理。错误示范是"什么任务都开子代理", 反而增加协调开销。

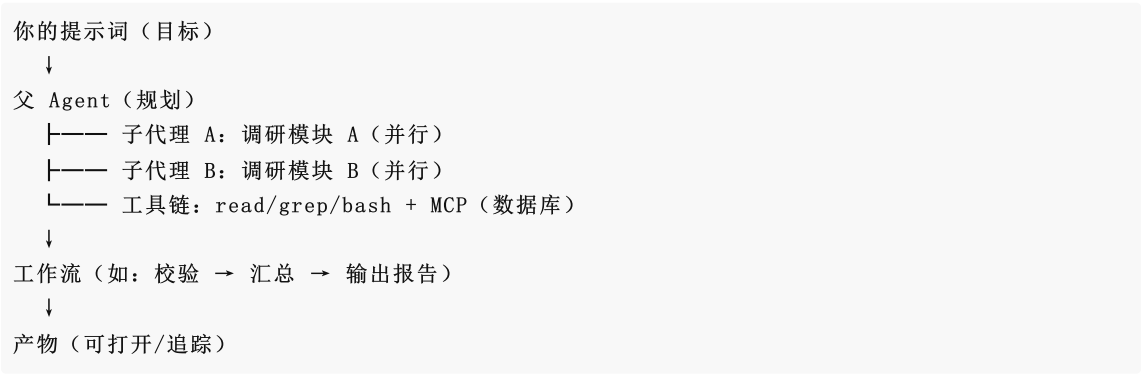
9.3 workflow (workflow) : 多步编排

是什么： `packages/workflow/*` 提供多步工作流编排（worker-thread provider + tool Consumer）。

和"多轮对话"的区别：普通对话是"模型自由发挥"，工作流是"把步骤定义成流程，按顺序/条件执行"——适合确定性流程（如：拉取数据 → 清洗 → 生成报表 → 校验）。

官方示例（ `packages/examples/` ）：有可运行的 `cordis.yml` 工作流示例。

9.4 组合起来：一个"Agent 系统"长什么样



新手路径建议：

1. 阶段一：单 Agent + 内置工具（第 2-5 章）
2. 阶段二：+ MCP（接外部工具）
3. 阶段三：+ 子代理（并行）
4. 阶段四：+ 工作流（确定性流程）

9.5 社区生态示例（2026-08-13 快照）

官方 Discussion 里已出现的社区项目：

- `dsh-plugin-cost-tracker` ——实时追踪 token 成本（插件/MCP 形态）
- 插件开发/提速类（如本白皮书配套的 `dsh-tool-turbo`）

生态正在快速生长——现在是进场搭积木的好时候。

动手练习（检验你是否真懂了）

1. 理解题：不看原文，说出 MCP、子代理、工作流各自解决什么问题。三者之间有什么联系？
自查：参考本章 9.1-9.3 节各节首段
2. 理解题：解释"工作流"和"多轮对话"的区别。什么场景该用工作流而不是让模型自由发挥？
自查：参考本章 9.3 节"和多轮对话的区别"段落
3. 动手题：在 `dsh web` 里发一个"调研当前仓库的目录结构并生成报告"的任务，观察模型是否自动调用了 `read / glob / grep` 等工具。如果想加一个"查数据库"的能力，应该用 MCP 还是写 host 插件？
自查：参考本章 9.1 节"MCP 是生态扩展" + 9.4 节组合示意图
4. 动手题：设计一个"子代理并行调研"的场景：假设你要调研一个仓库的 3 个模块（`src/lib/src/cli/src/web`），写出父 Agent 的规划 + 3 个子代理的任务描述
自查：参考本章 9.2 节"典型场景" + 9.4 节"Agent 系统"示意图
5. 思考题：本章 9.4 节给出"新手路径四阶段"。为什么建议"先用好单 Agent 再上子代理"？如果一上来就用子代理，可能遇到什么问题？

自查：参考本章 9.2 节"对新手"段落 + 9.4 节新手路径

6. 思考题：dsh-plugin-cost-tracker 是"追踪 token 成本"的插件/MCP。如果让你设计这个插件，你会用 host 插件还是 MCP？为什么？

自查：参考本章 9.1 节 MCP 定位 + 9.5 节社区生态示例

常见疑问 FAQ

Q1：MCP 是什么？和 dsh 插件有什么区别？MCP (Model Context Protocol) 是"给 Agent 插外部工具"的开放协议。dsh 插件是"在 dsh 运行时内注册能力" (host 半跑在 Node 进程)。MCP 服务器是独立进程，通过协议通信。区别：插件深度集成、性能好；MCP 解耦、可复用、适合接入现有系统。

Q2：子代理是怎么"并行"的？会不会有冲突？子代理由父 Agent 委派，各自独立执行任务 (packages/subagent/*)。并行时各子代理有自己的上下文和工具调用，不会互相干扰。但要注意：如果多个子代理改同一个文件，可能有冲突——建议任务分解时避免交叉。

Q3：工作流的"确定性流程"是什么意思？和模型自由发挥有什么区别？工作流把步骤定义成流程 (如 cordis.yml 配置)，按顺序/条件执行，每步做什么是指定的。多轮对话是"模型自由发挥"，每步做什么由模型决定。确定性流程适合"必须按固定步骤走"的场景 (如数据处理流水线)，自由发挥适合"探索性任务"。

Q4：我想接一个 MCP 服务器 (比如数据库)，具体怎么操作？ (以官方 changelog 为准，rc 阶段用法可能变) 一般步骤：① 安装 @deepseek-ai/dsh-mcp-client；② 在 profile 的 cordis.patch.yml 挂载 MCP 客户端插件；③ 配置 MCP 服务器地址/命令；④ 重启 dsh web。具体配置参考官方 packages/mcp/ 文档。

Q5：子代理和"开多个 dsh 进程"有什么区别？子代理是 dsh 内部的委派机制 (packages/subagent/*)，共享父 Agent 的上下文和会话。开多个 dsh 进程是完全独立的会话，互不知道对方。子代理适合"任务分解后并行执行 + 结果汇总"，多进程适合"完全独立的任务"。

Q6：本章说部分功能"待实测"，我怎么知道哪些功能已经可用？查官方仓库的 packages/AGENTS.md (架构文档) 和 packages/ 目录 (源码)。有完整源码 + 示例的功能基本可用；只有包名没有文档的可能还在开发中。也可以看官方 Discussions 的社区反馈。rc 阶段变化快，以官方 changelog 为准。

附录 A：术语表与命令速查

第 10 章：复杂实战案例 (dsh 真实跑出来的)

本章目标：用两个真实在 dsh 里跑完的复杂任务，展示 dsh 在多步工具链下的实际能力、产出质量与耗时。隐私声明：两个案例全部使用合成数据/自造代码，不涉及任何真实业务数据或敏感信息。

TL;DR (本章核心，30 秒版)

- 案例 A (数据质量分析，186 秒)：52 行脏数据 → 分析 → 清洗脚本 → 可视化 → 运行验证，52→35 行，所有问题归零
- 案例 B (5-bug 修复 + 49 测试，94 秒)：故意埋 5 个 bug 的计算器模块 → 全修复 → 49 个测试全过，覆盖边界/精度/异常
- dsh 的画像：多步工具链自动编排、有判断力 (主动说明数据权衡)、产物可追踪、耗时可控 (94-186s)
- 关键启示：提示词把验收标准写清楚 (如"运行验证")，dsh 会照着闭环——复杂任务建议明确"做什么 + 怎么验收"
- 失败恢复：本案例未触发；生产环境建议配 guard 超时 + 人工审批

案例 A：数据质量分析 + 清洗 + 可视化 (186 秒)

任务

给 dsh 一份含脏数据的合成 JSON (52 行: 缺失值、类型错误、重复行、异常值 99999) , 要求:

- 1. 分析数据质量问题
- 2. 编写清洗脚本 `clean.py`
- 3. 编写可视化脚本 `visualize.py` (生成 `chart.png`)
- 4. 运行两个脚本验证

dsh 怎么做的 (工具链)

```
read(sales_data.json) → write(clean.py) → bash(python clean.py)
→ write(visualize.py) → bash(python visualize.py) → read(输出) → 总结
```

产出与验证

产物	结果
<code>clean.py</code>	✅ 运行成功, 52 → 35 行, 所有问题归零
<code>visualize.py</code> + <code>chart.png</code>	✅ 生成有效 PNG (1950×825)
清洗策略	缺失金额中位数填充、异常值删除、去重、类型统一

 dsh 生成的销售额可视化 (案例 A 产物)

输出要点 (体现判断力, 不是机械执行)

- 中位数填充会在直方图产生 510 尖峰——插补的固有副作用, 建议说明
- 若 99999 是真实大额订单, 建议改用 IQR/分位数法而非一刀切删除
- 清洗后统计: $N=35$, 均值 489.43, 中位数 510, 标准差 181.73

dsh 不仅执行了任务, 还主动说明了数据处理的权衡与假设——这是 agent 工程层"会思考"的证据。

案例 B: 5-bug 代码修复 + 49 个测试 (94 秒)

任务

给 dsh 一个故意埋了 5 个 bug 的 Python 计算器模块, 要求: 找到所有 bug → 修复 → 编写完整单元测试 → 运行验证。

埋的 bug

- 1. `add` 误写为 `a - b`
- 2. `divide` 除零静默返回 0 (应抛异常)
- 3. `factorial` 负数静默返回 -1 (应抛异常)
- 4. `factorial` 的 `range(1, n)` 漏乘 `n`
- 5. `format_money` 未格式化为两位小数

dsh 的修复 + 测试 (验证: 49 passed)

完整测试见 [case-test-calculator.py](#)——49 个用例覆盖:

函数	覆盖
<code>add</code>	正/负/零/浮点/精度/交换律

subtract	负结果/零边界/浮点
multiply	符号组合/乘零/浮点
divide	整除/符号/浮点/除零必须抛 ZeroDivisionError
factorial	0!/1! 边界/已知值/负数必须抛 ValueError
format_money	补零/四舍五入/0.1+0.2 舍入/负数

```
python -m pytest test_calculator.py -v # 49 passed in 0.37s
```

观察：dsh 的表现

- 完整闭环：读 → 修 → 写测试 → 跑测试 → 总结，无人工干预
- 修复质量：5 个 bug 全修复且补了 docstring 说明
- 测试设计：覆盖边界（除零/负数/精度），不是"能跑就行"

案例总结：dsh 在复杂任务上的画像

维度	表现
多步工具链	✅ 自动编排（读→写→跑→验证→总结）
复杂任务耗时	94s-186s（同模型同网关，比 benchmark 简单任务慢但可控）
产出质量	✅ 有判断力（数据权衡说明、测试边界设计）
产物追踪	✅ 生成文件可在对话末尾直接打开
失败恢复	本案例未触发；生产建议配 guard 超时 + 人工审批

给新手的启示：dsh 处理"多文件、多步骤、要验证"的任务很顺手——这也是它作为 Agent 运行时的主要价值场景。
复杂任务建议：提示词把验收标准写清楚（如"运行验证"），dsh 会照着闭环。

动手练习（检验你是否真懂了）

- 理解题：不看原文，说出案例 A（数据质量分析）的工具链顺序（read → write → bash → ...），以及最终产出是什么
 📌 自查：参考本章"案例 A"的"dsh 怎么做的"段落
- 理解题：案例 B（5-bug 修复）里，dsh 补的 49 个测试覆盖了哪些边界情况？为什么"除零必须抛 ZeroDivisionError"是一个重要的测试点？
 📌 自查：参考本章"案例 B"的测试覆盖表格
- 动手题：设计一个类似的"数据质量分析"任务：准备一份含脏数据的 CSV（至少 3 种问题：缺失值、类型错误、异常值），写提示词让 dsh 分析 + 清洗 + 可视化，记录耗时和产出
 📌 自查：参考本章案例 A 的任务描述 + 产出验证表格
- 动手题：设计一个类似的"bug 修复"任务：写一个故意含 3 个 bug 的 Python 模块（如字符串处理、日期计算），让 dsh 找 bug → 修复 → 写测试 → 运行验证，记录耗时和测试通过率
 📌 自查：参考本章案例 B 的任务描述 + "dsh 的表现"观察段落
- 思考题：案例 A 里 dsh "主动说明了数据处理的权衡与假设"（如中位数填充的尖峰副作用）。这个行为对"agent 工程层"的设计有什么启示？模型只是执行命令，还是真的有"判断力"？

自查：参考本章案例 A 的“输出要点”段落 + 案例总结表格

6. 思考题：本章结语说“提示词把验收标准写清楚，dsh 会照着闭环”。如果案例 A 的提示词只说“处理一下这个数据”而没说“运行验证”，dsh 的表现会有什么不同？

自查：参考本章“给新手的启示”段落 + 案例 A 的工具链（含 bash 运行验证）

常见疑问 FAQ

Q1：案例 A 耗时 186 秒，案例 B 耗时 94 秒，这个速度算快还是慢？取决于任务复杂度和模型档位。同模型同网关下，比 benchmark 简单任务慢但可控。关键不是绝对速度，而是“多步工具链自动编排 + 产出质量 + 无人工干预”的综合价值。如果想提速，参考第 6 章的推理档位策略 + tool-turbo 插件。

Q2：案例里的“合成数据”是什么意思？为什么不用真实数据？隐私保护。本白皮书不涉及任何真实业务数据或敏感信息。合成数据是“故意构造的、不含真实信息的数据”，用于演示 dsh 的能力。你在自己环境里可以用真实数据，但要注意数据安全。

Q3：如果 dsh 在复杂任务中“卡住了”（比如某步工具调用失败），怎么办？几种处理：① 看 dsh 进程日志，定位哪步失败；② 检查工具调用的输入是否正确（如文件路径、命令语法）；③ 配 guard 超时（guard/* 包），避免无限等待；④ 生产环境建议配人工审批（interaction/*），危险操作前确认。

Q4：案例 B 的 49 个测试是 dsh 自动生成的，质量怎么样？质量不错。覆盖了边界（除零/负数/精度）、异常（必须抛特定异常）、常规（正/负/零/浮点）。但自动生成的测试可能漏掉“业务逻辑特有的边界”——建议人工 review 测试设计，补充领域特定的用例。

Q5：我想让 dsh 跑一个“多文件、多步骤”的任务，提示词怎么写效果最好？三个原则：① 明确目标（“做什么”）；② 明确验收标准（“怎么算完成”，如“运行验证”“49 个测试全过”）；③ 明确约束（“不能做什么”，如“不能删原始数据”）。参考本章两个案例的提示词结构。

Q6：案例总结说“失败恢复本案例未触发”，那 dsh 的失败恢复能力怎么样？dsh 有 compaction（上下文溢出恢复）、guard（超时）、interaction（审批）等机制，但复杂场景的失败恢复还在演进中。生产环境建议：① 配 guard 超时避免卡死；② 配人工审批避免误操作；③ 长任务分步验证，不要“一口气跑完再检查”。具体配置参考第 8 章 8.5 节安全模型。

附录 A：术语表与命令速查

扩展案例（第二批，三个功能领域）

完整执行报告见 [case-expansion-report.md](#)，产物在 `docs/assets/`。全部合成数据。

案例 C：HTML 解析（Web 前端，约 3m18s）

任务：给定含表格+表单+图表的合成 HTML，编写零第三方依赖的解析脚本提取表格并输出 CSV。

dsh 的表现：

- 零依赖权衡：主动选标准库 `html.parser`（而非 BeautifulSoup）
- 健壮性：实现表格嵌套深度计数，防止越界
- 路径无关：用 `__file__` 定位输入，不依赖运行目录

产物：[case-c-parser.py](#) | 验证：输出 9 行×7 列 CSV，与源表逐字段一致

案例 D：代码重构（软件工程，约 4m37s）

任务：把约 200 行过程式订单脚本（含重复代码 `calc_order_total` / `calc_order_total_v2`）重构为 OOP + 职责分离，行为一致，测试验证。

dsh 的表现：

- 精准识别重复：合并两份相同逻辑为 `Order.total()` 单一入口
- SOLID 设计：Product/Customer/Order（模型）+ OrderProcessor（查询）+ ReportGenerator（报告）
- 向后兼容：保留模块级函数接口，可直接替换
- 测试深度：17/17 PASS——不仅对比函数返回值，还对比脚本级整体运行产物（report.txt / orders.json）
- 沙箱感知：发现 `tempfile.mkdtemp` 在沙箱下 0700 ACL 不可写，主动换方案

产物：[case-d-orders_refactored.py](#) + [case-d-test_refactor.py](#) | 验证：17/17 PASS

案例 E：API 文档生成（文档，约 1m04s）

任务：给含 8 个函数的 Python 模块生成完整 Markdown API 文档（参数表/返回值/异常/示例/边界）。

dsh 的表现：

- 契约差异识别：主动发现“文档声明”与“代码实现”的差异（如 `user_id` 字符集未强制校验），如实写入边界章节
- 防御式文档：为每个函数标注调用陷阱（负数 limit、空 updates 行为）
- ReadTheDocs 风格：423 行，8 个函数全覆盖，示例含 doctest 风格

产物：[case-e-api.md](#)

第二批案例画像

维度	表现
跨领域能力	✅ Web/工程/文档三种任务都能闭环
工程意识	✅ 零依赖权衡、沙箱感知、向后兼容、契约差异识别
验证习惯	✅ 每个案例都运行验证（含脚本级产物对比）
耗时	1-5 分钟/案例，复杂度越高越久

总结：dsh 在“要写代码 + 要验证”的工程任务上表现稳定，且会做权衡（依赖/兼容/安全）——这是 agent 从“能答”到“能干活”的分水岭。

第 11 章：未来展望——dsh 与 Agent 生态的演进预测

本章目标：从多个角度推演 dsh 及其生态的未来——技术、生态、竞争、行业、开发者机会、风险。预测不是断言，是“基于当前事实的推演”，供你判断是否值得投入。

TL;DR（本章核心，30 秒版）

1. 短期（1-3 月）：正式版 0.1.0 落地、插件生态爆发、教程/工具类项目吃第一波红利
2. 中期（1 年）：dsh 成为“可编程 Agent 底座”的事实标准候选，垂直场景插件成熟
3. 长期（2-3 年）：Agent 工程层标准化，dsh 生态与模型迭代相互成就
4. 最大风险：破坏性变更、生态碎片化、官方策略转向
5. 个人机会：现在入场（教程/插件/工具）是低成本的早期红利

11.1 技术演进预测

时间	预测	依据
----	----	----

短期	0.1.0 正式版发布（去掉 rc），破坏性变更收敛	当前 rc.6，官方明示"将有破坏性变更"，迭代极快
短期	官方补上 TUI 与交互式 CLI（社区呼声最高，#172 15+ 评论）	官方 Discussion 最热需求
中期	缓存/性能优化成为主旋律（reasoning_effort 精细化、缓存命中率工具化）	高缓存命中率已是成本优势（第 5 章），会被工具化显性化
中期	插件生态标准化（插件市场/一键安装/质量分级）	官方已有 dsh-plugin topic，插件数量增长必然催生市场
长期	Agent 运行时架构沉淀为"工程范式"（类似当年 Linux 内核模式的 Agent 版）	"everything is a plugin" 已被验证为可组合架构

11.2 生态发展预测

- 插件数量：零日 → 1 年内预计数十到上百个（参照同类生态 S 曲线）
- 内容生态：教程/博客/awesome 列表先爆发（中文内容目前是真空——本白皮书是第一份系统教程）
- 社区规模：官方 Discussion 日均新增讨论（我们观察到单日 30+ 新话题），Discord 活跃度会同步增长
- 头部项目：会诞生几个"生态旗舰"（类似 VSCode 生态的 Prettier/ESLint 地位）——工具型 + 教程型最可能

11.3 竞争格局预测

维度	dsh 的定位演进
vs Claude Code	短期仍"开箱即用不如"，长期靠"可定制 + 开源 + 成本"差异化
vs OpenCode	都开源；dsh 的官方后端 + 插件生态是差异点（OpenCode 无官方后端）
vs 自建框架	dsh 是"半成品到成品的桥梁"——比自建省事，比闭源可控
模型侧	DeepSeek 模型每代迭代都会放大 dsh 生态价值（模型 + 运行时协同）

关键判断：dsh 的胜负手不在"模型多强"（那是 DeepSeek 的事），而在**"Agent 工程层能否成为标准"**——谁能定义"插件怎么写、生态怎么长"，谁就赢下这一层。

11.4 行业应用预测

- 金融/数据分析：长会话 + 高缓存命中 = 成本友好的批量分析，最先落地
- 企业知识库/文档：1M 上下文 + 文档工具链，企业内知识管理是天然场景
- 教育：批量出题/解析/归纳，成本敏感场景吃缓存红利
- 垂直 Agent 产品：会涌现"行业 Agent 封装"（在 dsh 上做行业插件/配置包）

11.5 开发者机会（现在入场的人）

角色	机会	时机
教程/内容作者	中文教程真空（白皮书是第一份）	现在（红利最大）
插件开发者	官方缺位方向：TUI、远程访问、移动端、缓存工具	现在
工具型项目	提速、耗时可视化、MCP 工具包	现在（我们已做 tool-turbo 验证）
生态基础设施	插件市场、模板生成器、评测工具	中期

行业解决方案	金融/教育等行业插件包	中期
--------	-------------	----

11.6 风险与不确定性（诚实版）

- 1. 破坏性变更：rc 阶段 API 可能频繁变动——插件/教程需持续跟进（白皮书会同步更新）
- 2. 生态碎片化：插件质量参差，缺乏标准——早期需要“高质量示范项目”（本白皮书 + tool-turbo 是示范之一）
- 3. 官方策略转向：官方可能收回某些方向（如内置 TUI）——生态项目要有差异化壁垒
- 4. 模型竞争：其他模型迭代可能削弱 DeepSeek 吸引力——但 dsh 模型无关的设计（可接 OpenAI 兼容模型）是缓冲
- 5. 热度回落：如果 dsh 生态没起来，早期投入可能“白费”——但教程/技能本身是可迁移资产

11.7 白皮书自己的展望

- 内容：随 dsh 迭代持续更新（版本号对齐 rc/正式版）
- 双语：英文版完成度追上中文
- 案例：更多行业真实案例（合规前提）
- 生态：与 dsh-tool-turbo、awesome 列表、官方 Discussion 联动，成为生态内容枢纽之一

11.8 给读者的最终建议

- 想试试：现在装起来玩（第 2 章），成本几乎为零
- 想投入：挑一个“官方缺位 + 你有积累”的方向（TUI/行业插件/教程）
- 想观望：等 0.1.0 正式版，但会错过生态早期红利——早期红利 ≈ 先发者的耐心

预测会错，但“现在入场的成本最低”这一点，大概率不会错。

附录 A：术语表与命令速查

附录 A：术语表与命令速查

术语表

术语	一句话解释
Harness	套在模型外的工程层：会话、工具、上下文、循环控制
dsh	DeepSeek Harness 的命令行名（dsh 命令）
Profile	一种可启动形态（web / headless / 自定义），= bundle 栈 + patch 层
Bundle	一组插件的集合（官方：dsh-base、dsh-web-app、dsh-headless）
Cordis	dsh 底层的插件容器（依赖注入、事件、生命周期）
插件（Plugin）	cordis 插件；可同时携带 host 半（Node）与 client 半（浏览器）
host 半 / client 半	插件在 Node 侧（工具/服务/事件）与浏览器侧（UI）的两副面孔
扩展点	官方提供的钩子（agent/request、settings、conversationEvents、slots）
agent/request waterfall	每步模型请求前可改配置的事件链（插件在此注入 reasoningEffort 等）
reasoning_effort	思考强度（low/high/max）——速度与质量的权衡旋钮

headless	一次性 CLI 任务模式 (dsh --profile headless "任务")
compaction	长对话上下文压缩
subagent	子代理 (并行委派任务)
MCP	模型上下文协议 (接入外部工具服务器)
workflow	多步确定性工作流编排
locations	工具返回的文件路径 (驱动产物追踪)
产物文件	模型创建/修改的文件 (对话末尾的可打开 chips)
sandbox	命令执行的隔离沙箱
reasoning_effort	同「思考强度」
rc	预发布版本 (0.1.0-rc.6) , 迭代快、可能有破坏性变更

命令速查

dsh 核心

```
dsh web # 启动 Web UI
dsh --profile headless "任务" # 一次性任务 (脚本/CI)
dsh --version # 版本
dsh --dump-config # 合成配置树
dsh plugin --profile <name> add <pkg> # 安装插件
```

环境

```
node --version # 需 ≥ 22
npm install -g @deepseek-ai/dsh # 全局安装
npx -y @deepseek-ai/dsh web # 免安装运行
```

排障

```
netstat -ano | findstr 3080 # 端口占用 (Windows)
taskkill /PID <pid> /F # 杀进程
cat ~/.dsh/settings.yaml # 全局配置 (模型/推理档位)
```

插件开发

```
# 挂载到 profile (web 为例)
# ① package.json 加依赖: "pkg": "link:C:\\path\\to\\pkg"
# ② cordis.patch.yml 加:
#   - insert:
#     - id: <插件id>
#       name: <npm包名>
cd ~/.dsh/profiles/web && pnpm install # 安装
```

配置参考 (settings.yaml)

```
agent-default-model:
  model: deepseek-v4-flash      # 或 deepseek-v4-pro
  reasoningEffort: high        # low / high / max
```

附录 B: 同模型多 Agent 实测

附录：同模型 × 不同 Agent 实测对比 (Benchmark)

实测日期: 2026-08-13 | 模型: deepseek-v4-flash (同一 opencode-go 网关, 同一 API key) | Agent: dsh / opencode / omp 目的: 控制模型变量, 对比 Agent 工程层的差异 (提示词、工具链、轮次管理)。

方法

项	说明
模型	deepseek-v4-flash (三 agent 均走 opencode-go 网关 + 同一 key, 公平对照)
Agent	dsh (headless profile)、opencode (opencode run)、omp (--print 非交互)
任务	T1 创建文件 / T2 搜索统计 / T3 修复 bug + 验证
环境	Windows 11 + Node 24, 独立工作目录, 无预置上下文
度量	耗时 (秒) + 正确性 (人工核对产物/输出)

结果 (3 轮采样, 取中位数)

Agent	T1 创建文件	T2 搜索统计	T3 修 bug+验证	总计 (中位)	正确率
omp	10s	11s	15s	36s	27/27
dsh	22s	15s	48s	85s	27/27
opencode	35s	37s	42s	114s	27/27

原始 3 轮 (秒) :

Agent	轮次	T1	T2	T3
dsh	1 / 2 / 3	11 / 27 / 22	11 / 15 / 22	27 / 48 / 74
opencode	1 / 2 / 3	18 / 35 / 35	19 / 37 / 37	50 / 41 / 42
omp	1 / 2 / 3	9 / 10 / 12	9 / 11 / 12	13 / 33 / 15

解读

- 1. 正确率三家持平 (27/27) ——同模型下, Agent 工程层的差异主要体现在效率而非能力上限 (对这三个任务而言)。
- 2. 总耗时稳定排序: omp < dsh < opencode (3 轮一致)。

3. 任务类型影响排序：T3（多步：读→改→运行验证）波动最大（dsh 27→74s），说明复杂任务对 Agent 的轮次管理/工具效率更敏感。
4. dsh 定位：dsh 居中，且 headless 是“运行时 + 插件生态”——同样的模型，通过插件（如 tool-turbo 降推理档）可进一步优化耗时（见第 6 章）。
5. 样本警示：3 轮 × 3 任务，同网关同 key，含网络抖动——方向可信，绝对值会随环境波动。

可复现

```
# 三个 agent 的命令（独立目录）
dsh --profile headless "任务"
opencode run "任务" --model opencode-go/deepseek-v4-flash
omp "任务" --model deepseek-v4-flash --print
```

完整任务定义与产物见本仓库 [benchmark/](#) 目录（规划中）。